

# Minimum Change $\neq$ Best Cleaning: Parallel and Incremental Error Detection under Integrity Constraints

JIAHUI CHEN, College of Computer Science, Nankai University, China

YU SUN\*, College of Computer Science, Nankai University, China

SHAOXU SONG, Tsinghua University, China

HAIWEI ZHANG, College of Computer Science, Nankai University, China

XIAOJIE YUAN, College of Computer Science, Nankai University, China

Erroneous data frequently arise in practical scenarios due to a variety of factors, severely degrading data quality and impeding downstream applications. A widely adopted strategy for error detection is to detect conflicts based on integrity constraints and identify the minimum number of errors, thereby ensuring that the remaining cells satisfy the constraints. However, the minimum change principle may not be applicable in practical scenarios, since errors can occur simultaneously or irregularly. Therefore, this study employs Bayesian statistics to identify erroneous attribute values in conflicting cells that violate inter-attribute dependencies, rather than simply relying on the minimum change principle. This approach ensures that our work neither misses multiple erroneous attribute values conflicting with each other nor mistakenly detects outliers without errors. Furthermore, to address the efficiency issues commonly encountered in constraint-based data cleaning methods, we design 1) parallel conflict detection and error determination methods with the guaranteed parallel scalability, and 2) efficient incremental error detection strategies that can also be executed in parallel with such guarantees. Experiments conducted on various datasets demonstrate the superiority of our error detection methods in terms of both effectiveness and efficiency.

CCS Concepts: • **Information systems** → **Data cleaning**.

Additional Key Words and Phrases: data cleaning; integrity constraints; error detection

## ACM Reference Format:

Jiahui Chen, Yu Sun, Shaoxu Song, Haiwei Zhang, and Xiaojie Yuan. 2025. Minimum Change  $\neq$  Best Cleaning: Parallel and Incremental Error Detection under Integrity Constraints. *Proc. ACM Manag. Data* 3, 4 (SIGMOD), Article 256 (September 2025), 26 pages. <https://doi.org/10.1145/3749174>

## 1 Introduction

Erroneous values are commonly observed in various real-world scenarios, owing to sensor failure, network transmission issues, etc [8]. These errors can have numerous adverse effects, impeding the execution of downstream applications and accurate decision-making. Therefore, it is crucial to effectively detect errors and eliminate the effects of them. As indicated in [23], in practice, the majority of users want to detect errors in the given data instance, but prohibit any modifications

---

\*Yu Sun is the corresponding author.

---

Authors' Contact Information: Jiahui Chen, College of Computer Science, Nankai University, China, 2110694@mail.nankai.edu.cn; Yu Sun, College of Computer Science, Nankai University, China, sunyu@nankai.edu.cn; Shaoxu Song, Tsinghua University, China, sxsong@tsinghua.edu.cn; Haiwei Zhang, College of Computer Science, Nankai University, China, zhhaiwei@nankai.edu.cn; Xiaojie Yuan, College of Computer Science, Nankai University, China, yuanxj@nankai.edu.cn.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/9-ART256  
<https://doi.org/10.1145/3749174>

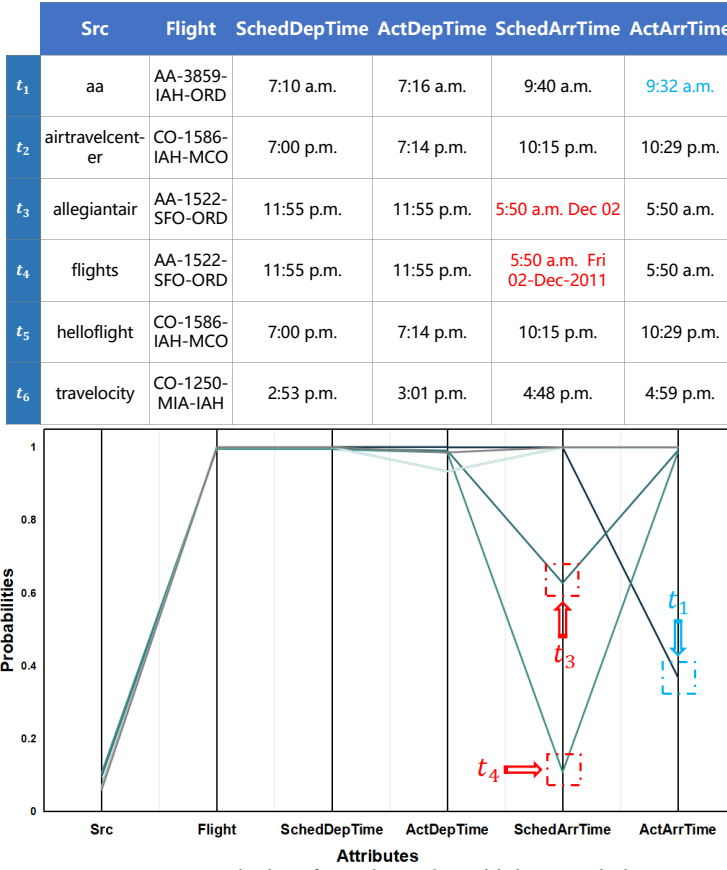


Fig. 1. Example data from the real-world dataset Flight

that could potentially harm their data. Consequently, numerous studies have been conducted focusing on the error detection problem.

With the surge in popularity of artificial intelligence, labeling-based error detection methods [24, 30, 34, 38] have recently been studied. While these methods can demonstrate good performance when the characteristics of training data and testing data are similar, this is not always guaranteed in practical applications. Furthermore, providing error labels is often very expensive. Statistics-based error detection methods [25, 37, 40, 46, 49] rely on the statistical information to identify errors, which often determine those abnormal values as erroneous ones. However, such a strategy may result in false positives and an excessive error set. Therefore, integrity constraints are the most widely used error detection techniques [17, 27, 43], since erroneous values and clean data will have conflicts w.r.t. the constraints. Statistical dependency-based methods like HoloClean [43] follow the minimum change principle, which may not align with real-world error patterns [44].

Recognizing the aforesaid problems, to effectively identify errors, we detect conflicts among tuples using constraints, and then perform error detection based on the dependency relationships and Bayesian statistics of attribute values. That is, for multiple cells in violation according to the constraints, we refer to their Bayesian statistics to further determine which are indeed erroneous, rather than only choosing one of them. The immediate benefit is that our study neither misses out multiple erroneous cells in violation with each other nor mistakenly identifies outliers that do not violate constraints as errors excessively.

Meanwhile, to address the efficiency issue in error detection, we propose a parallel error detection framework that substantially improves computational efficiency and can be naturally extended to incremental settings, avoiding full recomputation and enabling efficient error detection for incremental data. Specifically, our method parallelizes the key steps, including distance matrix construction, clustering, conflict detection, and final error determination, thereby reducing execution time without sacrificing accuracy. This parallel design enables the method to remain efficient across large-scale datasets, offering scalability as shown in experiments.

**EXAMPLE 1.** Consider some example data of the flight information in Figure 1, which are collected from the real-world Flight dataset used in experiments in Section 5. As shown,  $t_3[\text{SchedArrTime}]$  and  $t_4[\text{SchedArrTime}]$  are erroneously formatted, due to non-uniform data recording formats.

While statistics-based methods [25, 37, 40, 46, 49] can detect errors in  $t_3[\text{SchedArrTime}]$  and  $t_4[\text{SchedArrTime}]$ , they may misidentify rare but correct values (e.g.,  $t_1[\text{ActArrTime}]$ ). Our Bayesian statistics reduce false positives by focusing on constraint-violating cells. Consider a differential dependency (DD) [45]

$$\varphi_1 : \text{Flight} \leq 0.0, \text{SchedDepTime} \leq 0.0 \rightarrow \text{SchedArrTime} \leq 0.0,$$

equivalent to the functional dependency (FD) [36]  $\text{Flight}, \text{SchedDepTime} \rightarrow \text{SchedArrTime}$ . It denotes that if two tuples  $t_i, t_j$  have the same Flight and SchedDepTime, then they must also have the same SchedArrTime. Accordingly, tuples  $t_3$  and  $t_4$  have conflicts w.r.t. this constraint, since  $t_3[\text{Flight}] = t_4[\text{Flight}]$  and  $t_3[\text{SchedDepTime}] = t_4[\text{SchedDepTime}]$ , but  $t_3[\text{SchedArrTime}] \neq t_4[\text{SchedArrTime}]$ . Existing constraint-based methods [17, 27, 43] identify either  $t_3[\text{SchedArrTime}]$  or  $t_4[\text{SchedArrTime}]$  as the error, according to the minimum change principle. As shown in the lower part of Figure 1, the parallel coordinate graph visualizes the Bayesian statistics [32] of all the cells from the upper table. It presents the probability values for each tuple across different attributes. In this plot, the x-axis represents all the attributes of the tuples, and the y-axis denotes the corresponding probability values computed through Bayesian statistics. We observe that  $t_3$  and  $t_4$  are markedly different from the other tuples at the attribute SchedArrTime, as their probability values are significantly lower compared to those of the other tuples on this attribute. Therefore, in contrast to constraint-based methods, statistics-based methods and ours can effectively identify both  $t_3[\text{SchedArrTime}]$  and  $t_4[\text{SchedArrTime}]$  as erroneous.

Consider another DD, stating that any two tuples having different Flight and SchedArrTime will also have different ActArrTime.

$$\varphi_2 : \text{Flight} > 2.0, \text{SchedArrTime} > 3.0 \rightarrow \text{ActArrTime} > 0.0.$$

Notably,  $\varphi_1$  and  $\varphi_2$  are obtained from the Flight dataset using [31]. Therefore, since  $t_1$  has a different Flight value and its SchedArrTime and ActArrTime are also different from those of the other tuples, it satisfies the constraint  $\varphi_2$  with other tuples and can be regarded as a clean tuple. As shown in the parallel coordinate graph in Figure 1, the outlier is identified based on the conditional probability, not the marginal frequency. Since Flight and SchedArrTime are correlated with ActArrTime, the conditional probability of  $t_1[\text{ActArrTime}]$  given  $t_1[\text{Flight}]$  and  $t_1[\text{SchedArrTime}]$  is low, which leads to its classification as an outlier and causes it to be mistakenly detected as erroneous by statistics-based methods [25, 37, 40, 46, 49]. On the contrary, our methods will not make such mistakes, under the guidance of both integrity constraints and Bayesian statistics.

To detect conflicts and determine errors w.r.t.  $\varphi_1$  and  $\varphi_2$ , we know that all the attributes involved in  $\varphi_1$  and  $\varphi_2$  should be considered among tuple pairs. However, it is notable that, for each attribute, we can conduct the distance matrix construction, clustering, conflict detection, and error determination processes in parallel, instead of conducting them sequentially.

## 1.1 Challenges

To effectively and efficiently detect errors according to dependency relationships and integrity constraints is challenging.

(1) Although constraint-based error detection approaches have been widely studied with various constraints, it is generally time-consuming to detect conflicts and determine errors among different tuples w.r.t. the specific constraints, especially for large-scale data.

(2) Moreover, error detection methods based on the minimum change principle, which aim to minimally modify the dataset to restore consistency, often struggle to adapt to real-world scenarios, and tend to suffer from both false positives and false negatives, leading to unreliable detection accuracy. Therefore, it is not trivial to provide a quick and accurate decision about which attribute values are erroneous under the guidance of integrity constraints.

(3) In practical applications, new data are constantly generated as business operations proceed, and they may contain erroneous values, resulting in new conflicts. Re-running the algorithm from scratch every time new data arrive will be highly time-consuming, especially with growing data and increasing detection complexity. Therefore, it is necessary to design the incremental algorithm to detect conflicts and determine errors introduced by the newly arriving data.

## 1.2 Contributions

To tackle challenges above, we make the following contributions.

(1) *Error detection problem (Section 2)*. We formalize the error detection problem according to the Bayesian statistics under the guidance of integrity constraints. This design abandons the minimum change principle, which often fails in real-world scenarios, enabling more accurate identification of erroneous cells through probabilistic modeling.

(2) *Parallel error detection (Section 3)*. We design parallel error detection strategies, including parallel distance matrix construction, clustering, conflict detection, and error determination, with the parallel scalability guarantee. This ensures high efficiency and strong scalability on large datasets, enabling practical deployment in real-world systems.

(3) *Incremental parallel error detection (Section 4)*. We further advance our method into an incremental parallel version, consisting of incremental distance matrix construction, clustering, conflict detection, and error determination, to meet the challenge from incrementally incoming data.

(4) *Experiments (Section 5)*. We conduct the empirical study over erroneous datasets, compared with eleven baselines in different categories, verifying the effectiveness and efficiency of our error detection methods, which spend low time costs identifying the most accurate errors.

Table 1 lists the frequently used notations in this study.

## 2 Error Detection Problem

In this section, we introduce the terminology used in this manuscript and formally define the error detection problem.

### 2.1 Integrity Constraints

Consider a relational instance  $I = \{t_1, \dots, t_n\}$  over schema  $R = (A_1, \dots, A_m)$ , where each tuple  $t_i = \{t_i[A_1], t_i[A_2], \dots, t_i[A_m]\}$  collects  $m$  attribute values  $t_i[A_j]$ ,  $A_j \in R$ .

Integrity constraints are typically used to detect conflicts in the instance  $I$ . In this study, we use the differential dependency (DD) [45], owing to its high expressive power, i.e., generalizing several types of constraints, such as functional dependency (FD) [36], matching dependency (MD) [21], relaxed functional dependency (RFD) [14], and metric functional dependency (MFD) [28].

Table 1. Notations

| Symbol              | Description                                            |
|---------------------|--------------------------------------------------------|
| $I$                 | relational instance consisting of $n$ tuples           |
| $R$                 | relational schema over $m$ attributes                  |
| $t_i[A_j]$          | cell of $i$ -th tuple $t_i$ on $j$ -th attribute $A_j$ |
| $\text{dom}(A_j)$   | domain values of attribute $A_j$                       |
| $\text{Cell}(I)$    | set of all cells in $I$                                |
| $d_{A_j}(t_i, t_l)$ | distance between $t_i[A_j]$ and $t_l[A_j]$             |
| $\phi[A_j]$         | differential function on attribute $A_j$               |
| $D_j$               | distance matrix for attribute $A_j$                    |
| $C_j$               | cluster set for attribute $A_j$                        |
| $\Sigma$            | a set of differential dependencies $\varphi$           |
| $E$                 | set of erroneous cells                                 |
| $\mathcal{P}$       | the set of processors                                  |
| $p_i$               | $i$ -th processor                                      |
| $\Delta$            | symbol of incremental part                             |

Although the denial constraint (dc) [15] is widely used in data cleaning studies and can express the order relationships between attribute values (e.g.,  $t_1[A_j] > t_2[A_j]$ ), DC cannot express precise differences between values. DD extends traditional dependencies by capturing both equality and difference constraints, supporting diverse data types and flexible distance metrics, making them suitable for modeling complex relationships.

A differential function  $\phi[A_j]$  specifies a condition  $A_j \text{ op } \theta$ , where  $\text{op} \in \{\leq, >\}$  and  $\theta$  is a threshold. A tuple pair  $(t_i, t_l)$  satisfies  $\phi[A_j]$ , denoted as  $(t_i, t_l) \asymp \phi[A_j]$ , if the distance  $d_{A_j}(t_i, t_l)$  satisfies  $\text{op } \theta$ ; otherwise, it violates the differential function, denoted as  $(t_i, t_l) \not\asymp \phi[A_j]$ .

The differential function defined on an attribute subset  $X$  within the schema  $R$ ,  $X \subseteq R$ , is denoted as  $\phi[X]$ . It is constructed by taking the conjunction of differential functions for all attributes in  $X$ , expressed as  $\phi[X] = \bigwedge_{A_j \in X} \phi[A_j]$ . A tuple pair  $(t_i, t_l)$  satisfies  $\phi[X]$ , denoted as  $(t_i, t_l) \asymp \phi[X]$ , if  $(t_i, t_l) \asymp \phi[A_j]$  holds for every  $A_j \in X$ . Otherwise,  $(t_i, t_l) \not\asymp \phi[X]$ .

A differential dependency (DD) is expressed as  $\phi[X] \rightarrow \phi[A_j]$ , where  $\phi[X]$  is the differential function defined on a subset of left-hand-side (LHS) attributes  $X \subseteq R$ , and  $\phi[A_j]$  is the differential function defined on a right-hand-side (RHS) attribute  $A_j \in R \setminus X$ . A DD can be formally defined as

$$\varphi : \bigwedge \{\phi[A_l] \mid A_l \in X\} \rightarrow \phi[A_j], \quad X \subseteq R, A_j \in R \setminus X. \quad (1)$$

For any tuple pair  $(t_i, t_l)$ , it satisfies the differential dependency  $\varphi$ , denoted as  $(t_i, t_l) \models \varphi$ , if  $(t_i, t_l) \asymp \phi[X]$  implies  $(t_i, t_l) \asymp \phi[A_j]$ . Otherwise, when  $(t_i, t_l) \asymp \phi[X]$  and  $(t_i, t_l) \not\asymp \phi[A_j]$ , we have  $(t_i, t_l) \not\models \varphi$ . Notably, for any tuple pair  $(t_i, t_l) \not\asymp \phi[X]$ , it does not violate the differential dependency  $\varphi$  as well.

Given a relational instance  $I$  over schema  $R$ , we write  $I \models \varphi$  to represent that every tuple pair  $(t_i, t_j) \in I \times I$  satisfies  $(t_i, t_j) \models \varphi$ . Otherwise,  $I \not\models \varphi$ , indicating that there exists at least one tuple pair  $(t_i, t_j) \in I \times I$  such that  $(t_i, t_j) \not\models \varphi$ . For a set of differential dependencies  $\Sigma$ ,  $I \models \Sigma$  if and only if  $I$  satisfies all the dependencies in  $\Sigma$ , i.e.,  $\forall \varphi \in \Sigma, I \models \varphi$ . Otherwise,  $I \not\models \Sigma$ .

**EXAMPLE 2 (EXAMPLE 1 CONTINUED).** Consider again  $\varphi_1 : \text{Flight} \leq 0.0, \text{SchedDepTime} \leq 0.0 \rightarrow \text{SchedArrTime} \leq 0.0$  in Example 1. In the instance  $I = \{t_1, \dots, t_6\}$  in Figure 1, the tuple pair  $(t_3, t_4)$  satisfies differential functions  $\phi[\text{Flight}]$  and  $\phi[\text{SchedDepTime}]$ , indicating that  $(t_3, t_4) \asymp \phi[X]$ . However,  $d_{\text{SchedArrTime}}(t_3, t_4) > \theta_{\text{SchedArrTime}}$  where  $\theta_{\text{SchedArrTime}} = 0$ , implying  $(t_3, t_4) \not\asymp \phi[\text{SchedArrTime}]$ . Thus,  $(t_3, t_4)$  violates the DD constraint  $\varphi_1$ , i.e.,  $(t_3, t_4) \not\models \varphi_1$ .

## 2.2 Bayesian Statistics

The marginal probability of a value  $t_i[A_j]$  in attribute  $A_j$  is estimated by its frequency in the dataset:

$$P(t_i[A_j]) = \frac{|\{t_l \mid t_l[A_j] = t_i[A_j], t_l \in I\}|}{|I|}. \quad (2)$$

That is, the numerator represents the number of tuples which also have the same value  $t_i[A_j]$  in the attribute  $A_j$ , and the denominator is the total number of tuples in  $I$ .

The joint probability describes the probability of multiple attributes simultaneously taking specific values. For each dependent attribute  $A_j$ , all the other attributes can be treated as its determinant attributes  $\mathbf{X} \subseteq R$ . For instance, it can be simply the set of attributes  $\mathbf{X} = \{A_1, \dots, A_{j-1}, A_{j+1}, \dots, A_m\}$  or any other subset of  $R$ . The goal is to model the conditional probability of each attribute for all tuples in  $I$  given its corresponding determinant attributes,  $P(t_i[A_j] \mid t_i[\mathbf{X}])$ , which quantifies the dependency between the dependent attribute and its determinant attributes.

According to the Naive Bayes model [33], the marginal probability of determinant attributes  $P(t_i[\mathbf{X}])$  can be decomposed into the product of the marginal probabilities of all constituent attributes,

$$P(t_i[\mathbf{X}]) = \prod_{A_l \in \mathbf{X}} P(t_i[A_l]). \quad (3)$$

Similarly, the conditional probability of the determinant attributes, given the dependent attribute, can also be decomposed as

$$P(t_i[\mathbf{X}] \mid t_i[A_j]) = \prod_{A_l \in \mathbf{X}} P(t_i[A_l] \mid t_i[A_j]). \quad (4)$$

Using the Naive Bayes model to estimate the conditional probability of dependent and determinant attributes, the posterior probability  $P(t_i[A_j] \mid t_i[\mathbf{X}])$  can be gotten based on the Bayes' theorem [26],

$$P(t_i[A_j] \mid t_i[\mathbf{X}]) = \frac{P(t_i[\mathbf{X}] \mid t_i[A_j])P(t_i[A_j])}{P(t_i[\mathbf{X}])}. \quad (5)$$

In this equation,  $P(t_i[\mathbf{X}] \mid t_i[A_j])$  represents the probability of observing the determinant attribute values  $t_i[\mathbf{X}]$  given the dependent attribute value  $t_i[A_j]$ , which can be expanded according to Formula 4. Similarly,  $P(t_i[\mathbf{X}])$ , representing the marginal probability of the determinant attribute values  $t_i[\mathbf{X}]$ , can be expanded following Formula 3.  $P(t_i[A_j])$  is the prior probability of the dependent attribute  $A_j$ , representing the probability of  $t_i[A_j]$  independent of any determinant attributes.

**EXAMPLE 3 (EXAMPLE 1 CONTINUED).** Consider the instance  $I$  in Figure 1. According to Bayesian statistics, we can obtain the marginal probability  $P(t_1[\text{ActArrTime}]) = 0.256$  for the cell  $t_1[\text{ActArrTime}]$ . Then its posterior probability is  $P(t_1[\text{ActArrTime}] \mid t_1[\text{Flight}], t_1[\text{SchedArrTime}]) = 0.368$ , following Formula 5. We can obtain  $P(t_3[\text{SchedArrTime}] \mid t_3[\text{Flight}], t_3[\text{SchedDepTime}]) = 0.627$  and  $P(t_4[\text{SchedArrTime}] \mid t_4[\text{Flight}], t_4[\text{SchedDepTime}]) = 0.108$  as well.

## 2.3 Problem Statement

To eliminate the violations, the error cell set  $E$  is a subset of  $\text{Cell}(I)$  such that the remaining cells  $\text{Cell}(I) \setminus E$  satisfy the constraint set  $\Sigma$ .

**DEFINITION 1 (ERROR SET).** Given a set of integrity constraints  $\Sigma$  defined over schema  $R$ , and an instance  $I$ ,  $E \subset \text{Cell}(I)$  is an error set w.r.t.  $\Sigma$  if  $\text{Cell}(I) \setminus E \models \Sigma$ .

Although most constraint-based approaches utilize the minimum change principle [10], as analyzed in Example 1, it may not necessarily apply in real-world applications. Specifically, for two

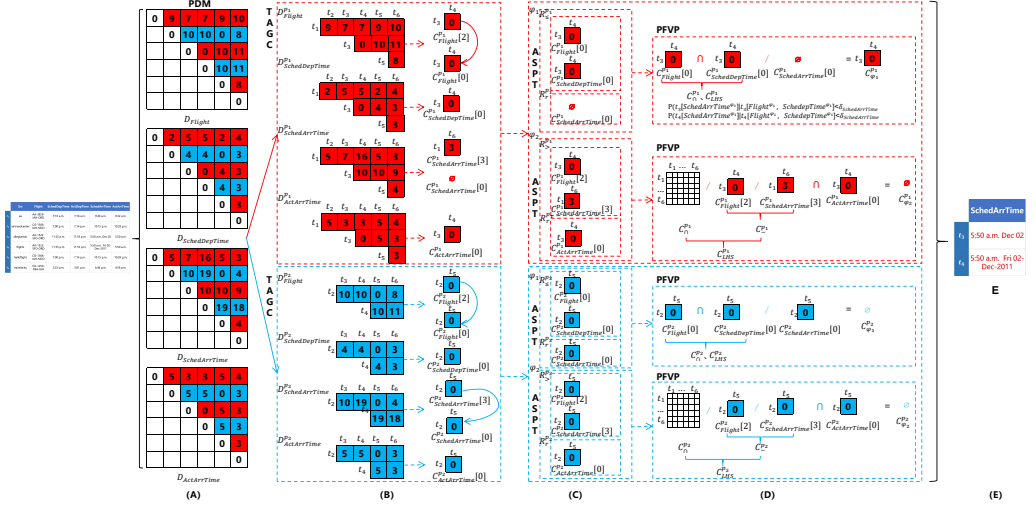


Fig. 2. An overview of our parallel error detection algorithm PED

tuples  $(t_i, t_l) \not\models \varphi$ , there may exist multiple cells  $t_i[A_x]$ ,  $t_i[A_j]$ ,  $t_l[A_x]$ ,  $t_l[A_j]$  contain erroneous values, only identifying any one of them is not accurate enough. In contrast, the conditional probability  $P(t_i[A_j] \mid t_i[X])$  in Formula 5 can better represent the dependency and cooccurrence relationships between different attribute values. For instance,  $t_i[A_x]$ ,  $t_i[A_j]$ ,  $t_l[A_x]$ ,  $t_l[A_j]$  each have a significantly lower probability than the other clean attribute values.

In this sense, given multiple cells (say  $t_i[X]$ ,  $t_l[X]$ ,  $t_i[A_j]$ ,  $t_l[A_j]$ ) involved into the conflict w.r.t. the constraint (say  $\phi[X] \rightarrow \phi[A_j]$ ), it is more reasonable to determine those erroneous cells  $t_i[A_j]$  having the lower probabilities  $P(t_i[A_j] \mid t_i[X])$ .

That is, given a relational instance  $I$  with the constraint set  $\Sigma$ , the error detection problem is to find an error set  $E$  of  $I$  w.r.t.  $\Sigma$ , such that the attribute value  $t_i[A_j] \in E$  has the lower probability  $P(t_i[A_j] \mid t_i[X]) < \delta_j$ , where  $\delta_j$  is a probability threshold for each attribute  $A_j \in R$ .

Each cell falls into only one of two cases: (i) if its probability is below the threshold, it is directly marked as an error, i.e.,  $\{t_i[A_j] \mid \exists t_i[A_j], P(t_i[A_j] \mid t_i[X]) < \delta_j\}$ ; (ii) otherwise, we check whether any conflicting cells (with this cell) belong to case (i). If so, errors are identified accordingly; otherwise, the cell with the lowest probability among the conflicting ones is marked as an error, i.e.,  $\{t_i[A_j] \mid \forall t[A], P(t[A] \mid t[X]) \geq \delta, \exists t_i[A_j], P(t[A] \mid t[X]) \geq P(t_i[A_j] \mid t_i[X])\}$ .

**EXAMPLE 4 (EXAMPLES 2 AND 3 CONTINUED).** As introduced in Example 2, we know  $(t_3, t_4) \not\models \varphi_1$ , where  $\varphi_1 : \text{Flight} \leq 0.0, \text{SchedDepTime} \leq 0.0 \rightarrow \text{SchedArrTime} \leq 0.0$ . Given the threshold  $\delta_{\text{ActArrTime}} = 0.1$  and  $\delta_{\text{SchedArrTime}} = 0.65$  (according to Figure 8), combining with the probabilities in Example 3, we know both  $P(t_3[\text{SchedArrTime}] \mid t_3[\text{Flight}], t_3[\text{SchedDepTime}]) < \delta_{\text{SchedArrTime}}$  and  $P(t_4[\text{SchedArrTime}] \mid t_4[\text{Flight}], t_4[\text{SchedDepTime}]) < \delta_{\text{SchedArrTime}}$ , but  $P(t_1[\text{ActArrTime}] \mid t_1[\text{Flight}], t_1[\text{SchedArrTime}]) > \delta_{\text{ActArrTime}}$ . Therefore,  $t_3[\text{SchedArrTime}]$  and  $t_4[\text{SchedArrTime}]$  are identified as errors, and  $t_1[\text{ActArrTime}]$  is not an error merely for being an outlier.

### 3 Parallel Error Detection

To tackle the first challenge (handling large-scale data) in Section 1.1, we develop the parallel algorithms for efficient conflict detection and error identification in this section, and show that they are parallelly scalable w.r.t. the number of processors (Proposition 1).

Specifically, as shown in Figure 2, given the instance  $I$  and the constraint set  $\Sigma$ , we study the parallel distance computation strategy in Section 3.1 (Procedure (A) in Figure 2), the cluster generation process in Section 3.2 (Procedure (B)), the parallel set operations in Section 3.3 (Procedure (C)), the conflict detection in parallel in Section 3.4 (Procedure (D)), and the parallel error identification in Section 3.5 (Procedure (E)).

### 3.1 Parallel Distance Matrix Calculation

In this section, we first present the algorithm  $\text{PDM}(I, \Sigma, \mathcal{P})$ , where  $\mathcal{P}$  is the set of processors, to parallelly compute the distance matrices  $\mathbf{D}$  of all tuple pairs in the instance  $I$  over the associated attributes  $R_\Sigma$  in the constraint set  $\Sigma$ . Specifically, for any differential dependency  $\varphi \in \Sigma$  with the left-hand-side (LHS) attributes  $X_\varphi$  and the right-hand-side (RHS) attribute  $A_\varphi$ , as introduced in Section 2.1, a differential function  $\phi[A_j]$  expresses  $A_j$  *op*  $\theta$ , where  $\theta$  is a threshold,  $A_j \in X_\varphi \cup \{A_\varphi\}$ . That is, we have  $R_\Sigma = \bigcup_{\varphi \in \Sigma} X_\varphi \cup \{A_\varphi\}$ , denoting the set of attributes related to the given constraint set  $\Sigma$ . Moreover, for each attribute  $A_j \in R_\Sigma$ , it may be involved in multiple differential functions  $\phi[A_j]$  in different constraints  $\varphi \in \Sigma$ . Let  $\Theta_j$  denote the set of thresholds associated with the attribute  $A_j$  in all constraints  $\varphi \in \Sigma$ , i.e.,  $\Theta_j = \{\theta_\varphi \mid \theta_\varphi \in \phi[A_j], \phi[A_j] \in \varphi, \varphi \in \Sigma\}$ .

To calculate the distance matrices  $\mathbf{D}$  over the associated attributes  $R_\Sigma$  in the constraint set  $\Sigma$ , we can compute the distance matrix  $D_j \in \mathbf{D}$  for each attribute  $A_j \in R_\Sigma$  respectively. That is, for any attribute  $A_j \in R_\Sigma$ , we consider the deduplicated domain  $\text{dom}(A_j)$ , i.e.,  $\forall t_i \in I, t_i[A_j] \in \text{dom}(A_j)$ , and  $\forall a_i \in \text{dom}(A_j), \nexists a_k \in \text{dom}(A_j)$ , such that  $a_i = a_k$ . Therefore, a total of  $\frac{|\text{dom}(A_j)| \cdot (|\text{dom}(A_j)| + 1)}{2}$  pairwise distances need to be computed, which can be processed in parallel across multiple processors. Considering all tuples  $t_i \in I$ , which may contain duplicate values in the attribute  $A_j$ , we can obtain its distance  $d_{A_j}(t_i, t_l)$  with any other tuple  $t_l \in I$ , according to  $\frac{|\text{dom}(A_j)| \cdot (|\text{dom}(A_j)| + 1)}{2}$  unique distance pairs in the domain  $\text{dom}(A_j)$ . In this way, we can obtain the distance matrices  $\mathbf{D}$  for all the attributes  $A_j \in R_\Sigma$  with the size of  $n \times n$ .

**EXAMPLE 5 (EXAMPLE 1 CONTINUED).** Consider the instance  $I$  and the constraint set  $\Sigma = \{\varphi_1, \varphi_2\}$  in Example 1. As shown in the Procedure (A) in Figure 2, PDM algorithm conducts the parallel distance matrix calculation based on the precomputed distances among deduplicated value pairs for each attribute, where  $\mathbf{D}$  consists of four individual distance matrices  $D_{\text{Flight}}, D_{\text{SchedDepTime}}, D_{\text{SchedArrTime}}, D_{\text{ActArrTime}}$ . Taking  $D_{\text{Flight}}$  as an example, as shown in the upper part of Figure 1, the deduplicated domain of Flight is  $\{\text{AA-3859-IAH-ORD}, \text{CO-1586-IAH-MCO}, \text{AA-1522-SFO-ORD}, \text{CO-1250-MIA-IAH}\}$ . Once the value-pair distances are precomputed,  $D_{\text{Flight}}$  can be efficiently constructed by looking up the corresponding distances, as it records the distances between each tuple pair on the Flight attribute. Due to the symmetry of the distance measure, only the upper triangular matrix is stored. To distribute the computation, we divide the row indices of the matrix by  $|\mathcal{P}|$ , and the number of processors  $|\mathcal{P}| = 2$  in our example. As shown, rows highlighted in red are assigned to processor  $p_1$ , while blue rows are assigned to processor  $p_2$ .

### 3.2 Generating Clusters

For each attribute  $A_j \in R_\Sigma$ , it may be involved in multiple differential functions  $\phi[A_j]$  and thresholds  $\theta_\varphi$  across different constraints  $\varphi \in \Sigma$ . Let  $\theta_j^{\max}$  denote the maximum threshold that the attribute  $A_j$  participates in within the constraint set  $\Sigma$ , i.e.,  $\theta_j^{\max} = \max_{\theta \in \Theta_j} \theta$ .

Algorithm 1 generates clusters for a given attribute  $A_j \in R_\Sigma$  by accepting its distance matrix  $D_j$ , the threshold set  $\Theta_j$ , as well as the start and end positions  $d_{ls}$  and  $d_{pq}$  in  $D_j$ . Specifically, the algorithm first creates the cluster for the maximum threshold  $\theta_j^{\max}$  by identifying all valid tuple pairs within the specified range of rows and columns that satisfy the threshold condition in Line 2. It then iteratively refines the clusters for the remaining thresholds in  $\Theta_j$  by filtering the previously



**Algorithm 1:** TAGC( $D_j, \Theta_j, d_{ls}, d_{pq}$ )

**Input:** Distance matrix  $D_j$ , the threshold set  $\Theta_j$ , the start position  $d_{ls}$  and the end position  $d_{pq}$  in  $D_j$

**Output:** Clusters  $C_j$

```

1 sort all thresholds  $\theta \in \Theta_j$  in decreasing order;
2  $C_j[\theta_j^{\max}] \leftarrow \{(t_i, t_u) \mid l \leq i \leq p, s \leq u \leq q, u > i, d_{iu} \leq \theta_j^{\max}\}$ ;
3 foreach  $\theta \in \Theta_j \setminus \{\theta_j^{\max}\}$  do
4    $C_j[\theta] \leftarrow \{(t_i, t_u) \mid (t_i, t_u) \in C_j[\theta_j^{\max}], d_{iu} \leq \theta\}$ ;
5    $\theta_j^{\max} \leftarrow \theta$ ;
6 return  $C_j$ ;
```

generated cluster in Line 2 and Line 4, based on the updated threshold in Line 5. Then the clusters for all the attributes  $R_\Sigma$  can be obtained by calling TAGC( $D_j, \Theta_j, d_{11}, d_{nn}$ ) for each attribute  $A_j \in R_\Sigma$ , which generates the cluster by iterating over the upper triangle of the distance matrix  $D_j$  and progressively refining the cluster for each threshold in  $\Theta_j$ .

**EXAMPLE 6 (EXAMPLE 5 CONTINUED).** After getting the distance matrices  $\mathbf{D}$  by calling PDM in the Procedure (A) in Figure 2, each processor further generates clusters following TAGC algorithm in the Procedure (B). Taking the processor  $p_1$  and the attribute Flight as an example. As discussed in Example 1, the attribute Flight appears in constraints  $\varphi_1$  and  $\varphi_2$  with the threshold set  $\Theta_{\text{Flight}} = \{\theta_{\varphi_1} = 0.0, \theta_{\varphi_2} = 2.0\}$ . To generate the cluster for the maximum threshold  $\theta_{\text{Flight}}^{\max} = \theta_{\varphi_2}$ ,  $p_1$  identifies all tuple pairs in its assigned portion of  $D_{\text{Flight}}$  that satisfy  $d_{\text{Flight}}(t_i, t_j) \leq 2.0$ . In this case, only the tuple pair  $(t_3, t_4)$  satisfies this condition, forming the cluster  $C_{\text{Flight}}^{p_1}[2]$ . Next, to refine the cluster for the lower threshold  $\theta_{\varphi_1}$ ,  $p_1$  filters  $C_{\text{Flight}}^{p_1}[2]$  by checking which tuple pairs satisfy  $d_{\text{Flight}}(t_i, t_j) \leq 0.0$ . Since  $d_{\text{Flight}}(t_3, t_4) \leq 0.0$ , the same tuple pair  $(t_3, t_4)$  forms the cluster  $C_{\text{Flight}}^{p_1}[0]$ .

### 3.3 Assigning Parallel Set Operations

Compared with traditional constraint-based methods, in our work, by using set operations such as intersections and differences to analyze constraints, we eliminate the need to enumerate every tuple pair for each attribute in the constraints.

**3.3.1 Sequential intersection and difference operations.** Unlike traditional conflict detection algorithms that conduct constraint checks for every tuple pair, we adopt a more efficient approach by analyzing constraints attribute by attribute. Specifically, for each attribute  $A_j \in R_\Sigma$ , the differential function  $\phi[A_j] = A_j \text{ op } \theta$  maps to the tuple pair set  $C_j[\theta]$  within clusters  $C_j$ , which is generated by Algorithm 1, through finding all valid tuple pairs  $(t_i, t_u) \in I \times I$  with distances  $d_{iu} \leq \theta$ .

To detect conflicts, it is equivalent to identify violating tuple pairs that satisfy the LHS functions  $\phi[X]$  but fail the RHS function  $\phi[A_j]$ . When the operator  $\text{op}$  in  $\phi[A_j] = A_j \text{ op } \theta$  is  $\leq$ , then  $C_j[\theta]$  will represent the tuple pairs satisfying  $\phi[A_j]$ . On the contrary, when  $\text{op}$  is  $>$ ,  $C_j[\theta]$  will contain the violating tuple pairs w.r.t.  $\phi[A_j]$ . According to the explanations in Section 2.1, only the tuple pairs satisfying the LHS differential functions  $\phi[X]$  need to be further evaluated to determine whether they violate the constraint  $\varphi$ .

Therefore, LHS and RHS attributes are processed separately. For any differential dependency  $\varphi \in \Sigma$ , since  $\phi[X]$  often consists of multiple differential functions with diverse operators such as  $\text{op} = \leq$  and  $\text{op} = >$ , we classify the LHS attributes into two types:  $R_{\leq}$  representing attributes where  $\text{op} = \leq$ , and  $R_{>}$  representing attributes where  $\text{op} = >$ . The RHS attribute, denoted as  $R_r$ , is treated

as another distinct category.

$$R_{\leq} = \{(A_l, \theta) \mid A_l \in X, \phi[A_l] = A_l \text{ op } \theta, \text{op} = \leq\}, \quad (6)$$

$$R_{>} = \{(A_l, \theta) \mid A_l \in X, \phi[A_l] = A_l \text{ op } \theta, \text{op} = >\}, \quad (7)$$

$$R_r = \{(A_j, \theta) \mid \phi[A_j] = A_j \text{ op } \theta, \phi[X] \rightarrow \phi[A_j]\}. \quad (8)$$

As evident from Formulas 6–8, the elements stored in these three sets are of the form  $(A_j, \theta)$ . This is because, in Algorithm 1, the clusters  $C_j$  generated for a given attribute  $A_j \in R_{\Sigma}$  include the results for each threshold  $\theta \in \Theta_j$ . This association facilitates the subsequent step of identifying violating tuple pairs by providing a clear mapping between attributes and their thresholds.

The operators  $\text{op}$  in the LHS differential functions  $\phi[X]$  can be either  $\leq$  or  $>$ , and set operations such as intersections ( $\cap$ ) or differences ( $\setminus$ ) are applied accordingly. Specifically, for  $\text{op} = \leq$ , the intersection operation is used to find the common tuple pair set  $C_{\cap}$  that satisfies multiple differential functions involving the attributes classified under  $R_{\leq}$ ,

$$C_{\cap} = \begin{cases} \bigcap_{(A_j, \theta) \in R_{\leq}} C_j[\theta], & R_{\leq} \neq \emptyset \\ \{(t_i, t_l) \mid t_i, t_l \in I, i < l\}, & R_{\leq} = \emptyset \end{cases} \quad (9)$$

where each attribute  $A_j$  corresponds to a tuple pair set  $C_j[\theta]$ , and the operator  $\text{op}$  in the function  $\phi[A_j] = A_j \text{ op } \theta$  is consistent with the condition used to extract  $C_j[\theta]$  from the distance matrix  $D_j$ .

On the other hand, as for the operator  $\text{op} = >$ , the tuple pair set  $C_{-}$  that violates the differential functions  $\{\phi[A_j] \mid A_j \in X, \phi[A_j] = A_j > \theta\}$  is defined as

$$C_{-} = \bigcup_{(A_j, \theta) \in R_{>}} C_j[\theta]. \quad (10)$$

Similarly, for this case, the tuple pair set  $C_j[\theta]$  for each attribute from these differential functions is derived from  $D_j$ , but the operator  $\text{op} = >$  in these differential functions is inconsistent with the condition used to form the corresponding set  $C_j[\theta]$ . By applying these operations, the tuple pair set  $C_{\text{LHS}}$  that satisfies the LHS differential functions of the constraint can be obtained,

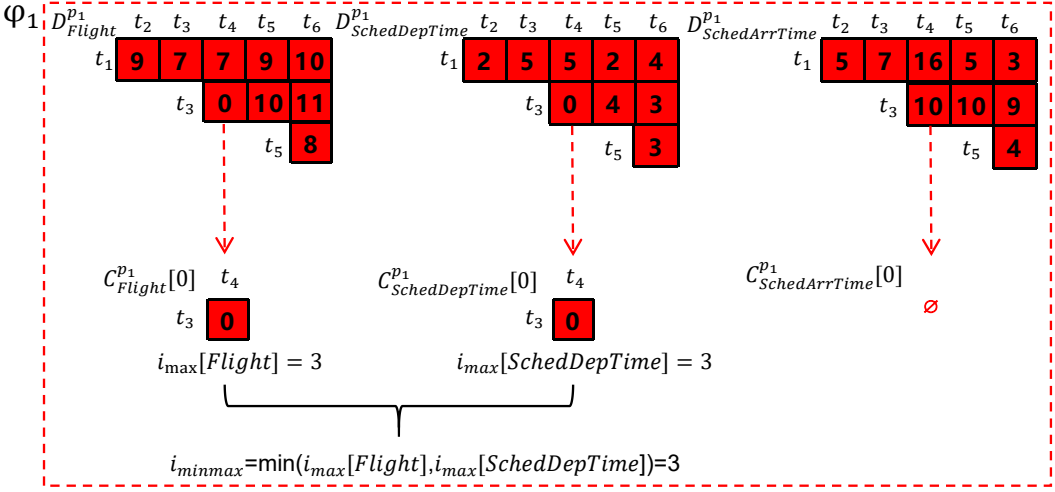
$$C_{\text{LHS}} = C_{\cap} \setminus C_{-}. \quad (11)$$

To obtain the final set of violating tuple pairs  $C_{\varphi}$ , the tuple pair set  $C_{\text{LHS}}$ , which satisfies the LHS functions  $\phi[X]$ , must be further refined using the tuple pair set  $C_j[\theta]$  included in  $R_r$ . This refinement is performed through intersection or difference operation depending on the operator  $\text{op}$  in the differential function  $\phi[A_j] = A_j \text{ op } \theta$ . Specifically, to ensure the tuple pair set violates the constraint  $\varphi$ , it is necessary to satisfy the LHS differential functions  $\phi[X]$  while violating the RHS differential function  $\phi[A_j]$ . Therefore, for the differential function  $\phi[A_j] = A_j \text{ op } \theta$ , the type of operation (intersection or difference) depends on  $\text{op}$ , which is the opposite of the operation used for the LHS differential functions, having

$$C_{\varphi} = \begin{cases} C_{\text{LHS}} \setminus C_j[\theta], & \text{op} = \leq \\ C_{\text{LHS}} \cap C_j[\theta], & \text{op} = > \end{cases} \quad (12)$$

This sequential process of performing intersection and difference operations ensures that the violating tuple pair set is identified based on the constraints attribute by attribute.

**EXAMPLE 7 (EXAMPLE 1 CONTINUED).** Consider the constraint  $\varphi_1$  in Example 1. For  $\varphi_1$ , the LHS differential functions involve the attributes Flight and SchedDepTime with  $\text{op} = \leq$ , forming  $R_{\leq} = \{(\text{Flight}, 0.0), (\text{SchedDepTime}, 0.0)\}$ , while there are no attributes with  $\text{op} = >$ , making  $R_{>}$  empty. The RHS differential function includes the attribute SchedArrTime, forming  $R_r = \{(\text{SchedArrTime}, 0.0)\}$ .

Fig. 3. The illustration figure of  $i_{\minimax}$ 

The set  $C_{\cap}^{p_1}$  is the intersection of the tuple pair sets  $C_{Flight}^{p_1}[0] = \{(t_3, t_4)\}$  and  $C_{SchedDepTime}^{p_1}[0] = \{(t_3, t_4)\}$ , thus  $C_{\cap}^{p_1} = \{(t_3, t_4)\}$ , as  $\theta = 0$  for both  $\phi[Flight]$  and  $\phi[SchedDepTime]$ . Since  $R_{>}$  is empty,  $C_{\cap}^{p_1} = \emptyset$ , and therefore  $C_{LHS}^{p_1} = C_{\cap}^{p_1} \setminus C_{\cap}^{p_1} = \{(t_3, t_4)\}$ . The violating tuple pair set  $C_{\phi_1}^{p_1}$  can be obtained by subtracting  $C_{SchedArrTime}^{p_1}[0] = \emptyset$  (corresponding to the RHS differential function  $\phi[SchedArrTime]$  with  $op = \leq$ ) from  $C_{LHS}^{p_1}$ ,  $C_{\phi_1}^{p_1} = \{(t_3, t_4)\}$ .

**3.3.2 Parallel intersection and difference operations.** Considering two attributes  $A_j, A_l \in R_{\Sigma}$ , their clusters  $C_j$  and  $C_l$  can be generated by calling Algorithm 1. If these two attributes appear in the same constraint  $\varphi \in \Sigma$  with differential functions  $\phi[A_j] = A_j \text{ op } \theta_j$  and  $\phi[A_l] = A_l \text{ op } \theta_l$ , the clusters  $C_j$  and  $C_l$  are distinguished by the row indices  $i$  of their respective distance matrices  $D_j$  and  $D_l$ . Only tuple pair sets with the same row index, such as  $\{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta_j]\}$  and  $\{(t_i, t_{j'}) \mid (t_i, t_{j'}) \in C_l[\theta_l]\}$ , can contain the same tuple pairs. Tuple pair sets with different row indices  $i \neq k$ , such as  $\{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta_j]\}$  and  $\{(t_k, t_{j'}) \mid (t_k, t_{j'}) \in C_l[\theta_l]\}$ , have no identical tuple pairs, and thus, operations such as intersection ( $\cap$ ) and difference ( $\setminus$ ) among these tuple pair sets are not meaningful.

As shown in Figure 3, to perform meaningful intersection and difference operations among tuple pair sets, it is essential to select a reference row index  $i_{\minimax}$ . For the intersection ( $\cap$ ) operation between  $C_j[\theta_j]$  and  $C_l[\theta_l]$ , the maximum row indices are  $i_{\max}[A_j] = \max_{(t_i, t_l) \in C_j[\theta_j]} i$  and  $i_{\max}[A_l] = \max_{(t_i, t_{j'}) \in C_l[\theta_l]} i$ , respectively. If  $i_{\max}[A_j] < i_{\max}[A_l]$ , then for any row index in  $(i_{\max}[A_j], i_{\max}[A_l])$ , there are no corresponding tuple pairs in  $C_j[\theta_j]$ , making the intersection within this range empty and the computation meaningless.

For the difference operation, if  $i_{\max}[A_j] > i_{\max}[A_l]$ , then for any row index in  $(i_{\max}[A_l], i_{\max}[A_j])$ , there are no corresponding tuple pairs in  $C_l[\theta_l]$ , so the tuple pairs in  $C_j[\theta_j]$  for this range remain unaltered. Conversely, if  $i_{\max}[A_j] < i_{\max}[A_l]$ , for any row index in  $(i_{\max}[A_j], i_{\max}[A_l])$ , there are no corresponding tuple pairs in  $C_j[\theta_j]$ , and the difference operation is meaningless in this range.

In summary, the reference index  $i_{\minimax}$  is chosen as the minimum one among the maximum row indices across tuple pair sets corresponding to the attributes involved in intersection operations,

$$i_{\minimax} = \min_{(A_j, \theta_j) \in R_{\leq}} i_{\max}[A_j] = \min_{(A_j, \theta_j) \in R_{\leq}} \max_{(t_i, t_l) \in C_j[\theta_j]} i, \quad (13)$$

which is the minimum row index among the maximum indices  $i_{\max}[A_j]$  of all attributes  $A_j, (A_j, \theta_j) \in R_{\leq}$ . This selection ensures that only the row indices up to  $i_{\minimax}$  are included, as

indices beyond this value lack corresponding tuple pairs in one or more sets used in the intersection, thereby pruning unnecessary rows.

**EXAMPLE 8 (EXAMPLE 6 CONTINUED).** In Figure 3, the selection of  $i_{\text{minimax}}$  for  $\varphi_1$  is illustrated. The LHS differential functions of  $\varphi_1$  involve the attributes Flight and SchedDepTime. To determine  $i_{\text{minimax}}$ , we consider the maximum row indices  $i_{\text{max}}$  from the tuple pair sets  $C_{\text{Flight}}^{p_1}[0] = \{(t_3, t_4)\}$  and  $C_{\text{SchedDepTime}}^{p_1}[0] = \{(t_3, t_4)\}$ . Specifically,  $i_{\text{max}}[\text{Flight}]$  is the maximum row index in  $C_{\text{Flight}}^{p_1}[0]$ , and  $i_{\text{max}}[\text{SchedDepTime}]$  is the maximum row index in  $C_{\text{SchedDepTime}}^{p_1}[0]$ . Since  $i_{\text{max}}[\text{Flight}] = 3$  and  $i_{\text{max}}[\text{SchedDepTime}] = 3$ , the reference row index is chosen as the minimum of these values, resulting in  $i_{\text{minimax}} = 3$ .

As explained in the determination of  $i_{\text{minimax}}$ , only clusters  $C_j[\theta_j]$  and  $C_l[\theta_l]$  for the attributes  $A_j$  and  $A_l$  in the same constraint  $\varphi \in \Sigma$  with differential functions  $\phi[A_j] = A_j \text{ op } \theta_j$  and  $\phi[A_l] = A_l \text{ op } \theta_l$  can contain the same tuple pairs if and only if the clusters share the same row index  $i$ , while clusters  $C_j[\theta_j]$  and  $C_l[\theta_l]$  with different row indices  $i$  and  $k$  are independent. To assign clusters  $C_j[\theta_j]$  and  $C_l[\theta_l]$  to different processors, the tuple pairs with the same row index  $i \leq i_{\text{minimax}}$  from these clusters must be assigned to the same processor to ensure consistency. Specifically, the tuple pair sets  $\{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta_j], i \leq i_{\text{minimax}}\}$  and  $\{(t_i, t_{j'}) \mid (t_i, t_{j'}) \in C_l[\theta_l], i \leq i_{\text{minimax}}\}$  should be assigned to the same processor, since they correspond to the same row index  $i$ .

We thus present the algorithm ASPT( $\varphi, C, i_{\text{minimax}}, \mathcal{P}$ ), which assigns the tuple pairs from clusters for all attributes in the constraint  $\varphi$  to processors  $\mathcal{P}$ , based on the reference row index  $i_{\text{minimax}}$ . For the constraint  $\varphi$ , we can divide its associate attributes into  $R_{\leq}$ ,  $R_{>}$ , and  $R_r$ , according to Formulas 6-8. For each processor  $p \in \mathcal{P}$ , ASPT algorithm can assign the subsets of clusters according to  $R_{\leq}$ ,  $R_{>}$ , and  $R_r$ , denoted as  $R_{\leq}^p$ ,  $R_{>}^p$ , and  $R_r^p$ . For a processor  $p$ , the assigned row indices of tuple pairs are defined as  $\{i \mid (i - p) \bmod |\mathcal{P}| = 0\}$ , ensuring that the processor only handles tuple pairs with row indices aligned to the intervals of  $\mathcal{P}$ ,

$$R_{\leq}^p = \{(A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta], i \leq i_{\text{minimax}}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_{\leq}\}, \quad (14)$$

$$R_{>}^p = \{(A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta], i \leq i_{\text{minimax}}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_{>}\}, \quad (15)$$

$$R_r^p = \{(A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in C_j[\theta], i \leq i_{\text{minimax}}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_r\}. \quad (16)$$

As shown, each of the above sets stores elements in the form  $(A_j, \theta, C_j^p[\theta])$ , where  $C_j^p[\theta]$  is the tuple pair subset derived from the cluster  $C_j[\theta]$  according to the attribute  $A_j$  and the threshold  $\theta$ . This structure allows the efficient retrieval of the specific tuple pair set  $C_j^p[\theta]$  based on  $A_j$  and  $\theta$  in the differential function  $\phi[A_j] = A_j \text{ op } \theta$  when identifying violating tuple pairs.

**EXAMPLE 9 (EXAMPLES 7 AND 8 CONTINUED).** Procedure (C) in Figure 2 illustrates the partitioning of tuple pairs for each attribute among processors. Taking  $\varphi_1$  as an example, based on the calculated  $i_{\text{minimax}}$ , all tuple pairs with row indices  $i \leq i_{\text{minimax}}$  and satisfying  $(i - 1) \bmod 2 = 0$  for the attributes Flight, SchedDepTime, and SchedArrTime are assigned to processor  $p_1$ . According to the sets  $R_{\leq}$ ,  $R_{>}$ , and  $R_r$  derived in Example 7, the tuple pair sets corresponding to the attributes in these sets are divided among processors, and those assigned to  $p_1$  are denoted as  $R_{\leq}^{p_1}$ ,  $R_{>}^{p_1}$ , and  $R_r^{p_1}$ . For  $\varphi_1$ ,  $R_{\leq}^{p_1}$  includes the attributes Flight and SchedDepTime, represented as  $\{(\text{Flight}, 0, \{(t_3, t_4)\}), (\text{SchedDepTime}, 0, \{(t_3, t_4)\})\}$ . Since  $R_{>}$  for  $\varphi_1$  is empty,  $R_{>}^{p_1}$  contains no tuple pairs. Similarly,  $R_r^{p_1}$ , which contains

the attribute and tuple pairs corresponding to the RHS differential function, has empty tuple pair set because no valid tuple pairs exist for  $C_{\text{SchedArrTime}}^{P_1}[0] = \emptyset$  within  $i \leq i_{\text{minimax}}$ .

### 3.4 Detecting Conflicts Parallely

After assigning tasks into different processors, we then design the algorithm  $\text{PFVP}(R_{\leq}^p, R_{>}^p, R_r^p)$  to detect conflicts in different processors parallely. Specifically, to identify the tuple pairs that satisfy the LHS differential functions  $\phi[X]$ , the attributes in  $R_{\leq}^p$  and  $R_{>}^p$  are processed separately. The operator of attributes in  $R_{\leq}^p$  is  $\leq$ , and each element in  $R_{\leq}^p$  is of the form  $(A_j, \theta, C_j^p[\theta])$ . Here,  $C_j^p[\theta]$  represents the tuple pair subset derived from the cluster  $C_j[\theta]$ , which satisfies the differential function  $\phi[A_j] = A_j \text{ op } \theta$  with  $\text{op} = \leq$ . Conversely, the operator of attributes in  $R_{>}^p$  is  $>$ , and the tuple pair subset  $C_j^p[\theta]$  in  $(A_j, \theta, C_j^p[\theta]) \in R_{>}^p$  does not satisfy the differential function.

By performing the intersection operation on the tuple pair sets within  $R_{\leq}^p$ , we compute the set  $C_{\cap}^p$  for the processor  $p \in \mathcal{P}$ , which contains all tuple pairs that satisfy the LHS functions with  $\text{op} = \leq$ . Subsequently, the difference operation is performed between  $C_{\cap}^p$  and the union of tuple pair sets in  $R_{>}^p$ , resulting in the final set  $C_{\text{LHS}}^p$  for the processor  $p \in \mathcal{P}$  that satisfies the LHS functions  $\phi[X]$  of the constraint  $\phi$ . Then Formulas 9-11 are refined accordingly,

$$C_{\cap}^p = \begin{cases} \bigcap_{(A_j, \theta, C_j^p[\theta]) \in R_{\leq}^p} C_j^p[\theta], & R_{\leq}^p \neq \emptyset \\ \{(t_i, t_l) \mid t_i, t_l \in I, i < l, (i - p) \bmod |\mathcal{P}| = 0\}, & R_{\leq}^p = \emptyset \end{cases} \quad (17)$$

$$C_{\text{LHS}}^p = C_{\cap}^p \setminus \bigcup_{(A_j, \theta, C_j^p[\theta]) \in R_{>}^p} C_j^p[\theta]. \quad (18)$$

To obtain the tuple pair set  $C_{\phi}^p$  for the processor  $p \in \mathcal{P}$  that violates the constraint  $\phi$ , we further refine  $C_{\text{LHS}}^p$  using the  $R_r^p$ , which corresponds to the differential function  $\phi[A_j] = A_j \text{ op } \theta$  on the RHS of the constraint  $\phi$ . The goal is to identify violating tuple pairs by ensuring they satisfy the LHS differential functions while violating the RHS function. For the  $(A_j, \theta, C_j^p[\theta]) \in R_r$ , the refinement operation depends on the  $\text{op}$  in the RHS function. Specifically, if  $\text{op} = \leq$ , the difference operation is performed between  $C_{\text{LHS}}^p$  and  $C_j^p[\theta]$ , i.e.,  $C_{\phi}^p = C_{\text{LHS}}^p \setminus C_j^p[\theta]$ . Conversely, if  $\text{op} = >$ , the intersection operation is applied between  $C_{\text{LHS}}^p$  and  $C_j^p[\theta]$ , i.e.,  $C_{\phi}^p = C_{\text{LHS}}^p \cap C_j^p[\theta]$ . This ensures that  $C_{\phi}^p$  contains only the tuple pairs that satisfy the LHS differential functions  $\phi[X]$  while simultaneously violating the RHS function  $\phi[A_j]$  of the constraint  $\phi$ .

**EXAMPLE 10 (EXAMPLE 9 CONTINUED).** *Procedure (D) in Figure 2 illustrates the parallel conflict detection process. Taking  $\phi_2$  on  $p_1$  as an example, from Procedure (C),  $\phi_2$  is assigned  $R_{\leq}^{p_1} = \emptyset$ ,  $R_{>}^{p_1} = \{(\text{Flight}, 2.0, \{(t_3, t_4)\}), (\text{SchedArrTime}, 3.0, \{(t_1, t_6)\})\}$ , and  $R_r^{p_1} = \{(\text{ActArrTime}, 0.0, \{(t_3, t_4)\})\}$ . Since  $R_{\leq}^{p_1}$  is empty, we cannot directly compute  $C_{\cap}^{p_1}$ . Instead, we generate the set of all tuple pairs between  $I^{p_1} = \{t_1, t_3, t_5\}$  and  $I = \{t_1, \dots, t_6\}$  as  $C_{\cap}^{p_1}$ . Next, we compute  $C_{\cap}^{p_1}$  by performing the union operation on the tuple pair sets for the attributes in  $R_{>}^{p_1}$ . Specifically, we compute  $C_{\text{Flight}}^{p_1}[2] \cup C_{\text{SchedArrTime}}^{p_1}[3]$ , where  $C_{\text{Flight}}^{p_1}[2] = \{(t_3, t_4)\}$  and  $C_{\text{SchedArrTime}}^{p_1}[3] = \{(t_1, t_6)\}$ . The result is  $C_{\cap}^{p_1} = \{(t_3, t_4), (t_1, t_6)\}$ . To compute  $C_{\text{LHS}}^{p_1}$ , we perform the difference operation between  $C_{\cap}^{p_1}$  and  $C_{\cap}^{p_1}$ . Thus,  $C_{\text{LHS}}^{p_1} = I^{p_1} \times I - \{(t_3, t_4), (t_1, t_6)\}$ . Therefore,  $C_{\text{LHS}}^{p_1} = \{(t_i, t_j) \mid (t_i, t_j) \in I^{p_1} \times I, (t_i, t_j) \notin \{(t_3, t_4), (t_1, t_6)\}\}$ . Finally,  $C_{\phi_2}^{p_1}$  is obtained by refining  $C_{\text{LHS}}^{p_1}$  using the RHS differential function  $\phi[\text{ActArrTime}]$ . This involves performing the intersection operation between  $C_{\text{LHS}}^{p_1}$  and  $C_{\text{ActArrTime}}^{p_1}[0]$ , where  $C_{\text{ActArrTime}}^{p_1}[0] = \{(t_3, t_4)\}$ . The result is  $C_{\phi_2}^{p_1} = C_{\text{LHS}}^{p_1} \cap C_{\text{ActArrTime}}^{p_1}[0] = \emptyset$ . Thus, the final set of violating tuple pairs for  $\phi_2$  on  $p_1$  is  $C_{\phi_2}^{p_1} = \emptyset$ .*

**Algorithm 2:** PED( $I, \Sigma, \mathcal{P}$ )**Input:** Instance  $I$ , constraint set  $\Sigma$ , and processors  $\mathcal{P}$ **Output:** Error set  $E$ 


---

```

1  $\mathbf{D} \leftarrow \text{PDM}(I, \Sigma, \mathcal{P});$ 
2 foreach  $A_j \in R_\Sigma$  do
3    $C_j \leftarrow \text{TAGC}(D_j, \Theta_j, d_{11}, d_{nn});$ 
4 foreach  $\varphi \in \Sigma$  do
5   foreach  $p \in \mathcal{P}$  in parallel do
6      $R_{\leq}^p, R_{>}^p, R_r^p \leftarrow \text{ASPT}(\varphi, C, i_{\text{minimax}}, \mathcal{P});$ 
7      $C_\varphi^p \leftarrow \text{PFVP}(R_{\leq}^p, R_{>}^p, R_r^p);$ 
8     foreach  $(t_i, t_l) \in C_\varphi^p$  do
9       foreach  $A_l \in A_j \cup X$  do
10        if  $P(t_i[A_l] \mid t_l[(A_j \cup X) \setminus A_l]) < \delta_j$  then
11           $E_\varphi^p \leftarrow E_\varphi^p \cup \{t_i[A_l]\};$ 
12        if  $P(t_l[A_l] \mid t_l[(A_j \cup X) \setminus A_l]) < \delta_j$  then
13           $E_\varphi^p \leftarrow E_\varphi^p \cup \{t_l[A_l]\};$ 
14        if  $E_\varphi^p = \emptyset$  then
15           $E_\varphi^p \leftarrow \{\min_{P(t[A_l] \mid t[(A_j \cup X) \setminus A_l])} t[A_l]\};$ 
16         $E \leftarrow E \cup E_\varphi^p;$ 
17         $E_\varphi^p \leftarrow \emptyset;$ 
18 return  $E$ 

```

---

**3.5 Putting Techniques Together**

We now present the complete procedure PED( $I, \Sigma, \mathcal{P}$ ) to detect conflicts in the instance  $I$  w.r.t. the constraint set  $\Sigma$  using processors  $\mathcal{P}$  in parallel. Line 1 first calculates the distance matrices  $\mathbf{D}$  for all attributes  $R_\Sigma$  by calling the algorithm PDM in Section 3.1. For each attribute  $A_j \in R_\Sigma$ , Algorithm 1 is then invoked in Line 3 to generate clusters  $C_j$  using the distance matrix  $D_j$ , the threshold set  $\Theta_j$ , the start position  $d_{11}$  and the end position  $d_{nn}$ . For each constraint  $\varphi \in \Sigma$ , attributes are classified into  $R_{\leq}$ ,  $R_{>}$ , and  $R_r$  based on Formulas 6-8. In Line 6, the algorithm ASPT assigns  $R_{\leq}^p$ ,  $R_{>}^p$ , and  $R_r^p$  to different processors  $\mathcal{P}$  based on the reference row index  $i_{\text{minimax}}$ , as defined in Formula 13. Each processor  $p \in \mathcal{P}$  computes the conflict tuple pairs  $C_\varphi^p$  w.r.t. the constraint  $\varphi \in \Sigma$  in Line 7 using the PFVP algorithm. Finally, in Lines 9-13, the algorithm checks whether any cell in the conflict tuple pair  $(t_i, t_l)$  has a probability  $P(t_i[A_l] \mid t_l[(A_j \cup X) \setminus A_l])$  below the threshold for the attribute  $A_j$ . If the condition is satisfied, the cell is added into the error set  $E_\varphi^p$ . Otherwise, the cell with the minimum probability is identified as an error in Line 15. Since each processor handles its own subset of conflicting tuple pairs, the error identification process is executed in parallel across processors, enabling efficient resolution. Finally, the results from all processors are merged to obtain the final error set  $E$  in Line 16.

**PROPOSITION 1.** *The complexity of the PED Algorithm is  $O\left(\frac{|\Sigma| \cdot m \cdot n^2}{|\mathcal{P}|}\right)$ .*

Please see the proofs for all the theoretical results in [1].

**EXAMPLE 11 (EXAMPLES 5, 6, 9, 10 CONTINUED).** *We illustrate the PED algorithm using  $\varphi_1$  as an example, referring to Figure 2. In Line 1, Procedure (A) computes distance matrices  $\mathbf{D}$  in Example 5. Then Lines 2-3 generate clusters using TAGC in Procedure (B), as explained in Example 6 to obtain  $C_{\text{Flight}}^{p_1}[0]$  and  $C_{\text{Flight}}^{p_1}[2]$  for the processor  $p_1$  on the attribute Flight. In Line 6, Procedure (C) assigns*

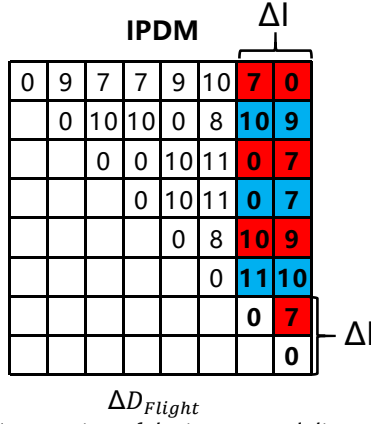


Fig. 4. An overview of the incremental distance matrix

$R_{\leq}^{p_1}, R_{>}^{p_1}, R_r^{p_1}$  to processor  $p_1$  for  $\varphi_1$  in Example 9. Then Procedure (D) uses PFVP to compute violating tuple pairs  $C_{\varphi_1}^{p_1}$  from  $R_{\leq}^{p_1}, R_{>}^{p_1}$  and  $R_r^{p_1}$  in Line 7 as explained in Example 10. Finally, in Lines 9-13, for  $\varphi_1$ ,  $P(t_3[\text{SchedArrTime}] \mid t_3[\text{Flight}], t_3[\text{SchedDepTime}]) = 0.627$  and  $P(t_4[\text{SchedArrTime}] \mid t_4[\text{Flight}], t_4[\text{SchedDepTime}]) = 0.108$  in Example 3 are both below  $\delta_{\text{SchedArrTime}} = 0.65$  (according to Figure 8), yielding the error set  $E = \{t_3[\text{SchedArrTime}], t_4[\text{SchedArrTime}]\}$ .

## 4 Incremental Parallel Error Detection

### 4.1 Incremental Error Detection Problem

Given a relational instance  $I$  over schema  $R$  with a set of constraints  $\Sigma$ , for the incremental updates  $\Delta I$ , the incremental error detection problem is to find an error set  $\Delta E$  for  $I \cup \Delta I$  w.r.t.  $\Sigma$ , such that the attribute value  $t_i[A_j] \in \Delta E$  has a conditional probability  $P(t_i[A_j] \mid t_i[X]) < \delta_j$ , where  $\delta_j$  is the threshold for each attribute  $A_j \in R$ . Specifically, if all the involved cells  $t_i[A_j]$  (w.r.t. the constraint  $\varphi$ ) have conditional probabilities above their respective thresholds, i.e.,  $P(t_i[A_j] \mid t_i[X]) \geq \delta_j$ , the cell with the minimum probability is selected as the erroneous one.

EXAMPLE 12. As shown in Figure 4, the instance  $I = \{t_1, \dots, t_6\}$  is incrementally updated with two new tuples:  $t_7 = \{\text{orbitz}, \text{AA-1522-SFO-ORD}, 11:55 \text{ p.m.}, 11:55 \text{ p.m.}, 5:50 \text{ a.m.}, 5:50 \text{ a.m.}\}$  and  $t_8 = \{\text{flights}, \text{AA-3859-IAH-ORD}, 7:10 \text{ a.m.}, 7:16 \text{ a.m.}, 9:40 \text{ a.m.}, 9:32 \text{ a.m.}\}$ , forming the incremental instance  $\Delta I = \{t_7, t_8\}$ . After adding  $\Delta I$  to  $I$ , new violating tuple pairs are detected:  $(t_3, t_7)$  and  $(t_4, t_7)$ . The conditional probability  $P(t_7[\text{SchedArrTime}] \mid t_7[\text{Flight}], t_7[\text{SchedDepTime}]) = 0.003$  is below the threshold  $\delta_{\text{SchedArrTime}} = 0.65$ , indicating that  $t_7[\text{SchedArrTime}]$  is an erroneous cell. Thus, the optimal incremental error set for the updated instance  $I \cup \Delta I$  is  $\Delta E = \{t_7[\text{SchedArrTime}]\}$ .

### 4.2 Incrementally Calculating Distance Matrices

Given the instance  $I$  with the corresponding distance matrices  $\mathbf{D}$ , for the new coming incremental instance  $\Delta I$ , we first design the algorithm  $\text{IPDM}(I, \Delta I, \Sigma, \mathcal{P})$  to compute the incremental distance matrices  $\Delta \mathbf{D}$  for it. Specifically, for each attribute  $A_j \in R_\Sigma$ , we calculate its incremental distance matrix  $\Delta D_j \in \Delta \mathbf{D}$  by considering the incremental domain of  $A_j$  in  $\Delta I$ , denoted as  $\Delta \text{dom}(A_j)$ , having  $\forall t_i \in I \cup \Delta I, t_i[A_j] \in \text{dom}(A_j) \cup \Delta \text{dom}(A_j)$ , and  $\forall a_i \in \Delta \text{dom}(A_j), \nexists a_k \in \Delta \text{dom}(A_j), \nexists a_l \in \text{dom}(A_j)$ , such that  $a_i = a_k$  or  $a_i = a_l$ . That is, we calculate distances between values in  $\Delta \text{dom}(A_j)$  and  $\text{dom}(A_j)$ , as well as distances within  $\Delta \text{dom}(A_j)$ , resulting in almost  $|\Delta \text{dom}(A_j)| \cdot (|\Delta \text{dom}(A_j)| + |\text{dom}(A_j)|)$  unique value pairs. These computations are parallelized across processors in  $\mathcal{P}$ . For tuple pair distance computation, we calculate distances between all tuples  $t_i \in I$  and  $\Delta t_j \in \Delta I$ , as

well as within all tuples  $\Delta t_j, \Delta t_l \in \Delta I$ . Then the tuple pair distances between any  $t_i, t_l \in I \cup \Delta I$  can be incrementally obtained accordingly.

**EXAMPLE 13.** As shown in Figure 4, it presents the incremental distance matrix after adding  $\Delta I = \{t_7, t_8\}$  to the original instance  $I = \{t_1, \dots, t_6\}$ . Instead of recalculating the entire distance matrices for  $I \cup \Delta I$ , IPDM algorithm incrementally computes the distance matrices  $\Delta D$  for the attributes in  $\varphi_1$ , namely Flight, SchedDepTime, and SchedArrTime. For each attribute  $A_j$ , the incremental distance matrix  $\Delta D_j$  is computed based on distances between values in  $\Delta \text{dom}(A_j)$  and  $\text{dom}(A_j)$ , as well as within  $\Delta \text{dom}(A_j)$ . For example,  $\Delta D_{\text{Flight}}$  includes distances between  $\{t_7[\text{Flight}], t_8[\text{Flight}]\}$  and  $\{t_1[\text{Flight}], \dots, t_6[\text{Flight}]\}$ , and within  $\{t_7[\text{Flight}], t_8[\text{Flight}]\}$ . Similarly,  $\Delta D_{\text{SchedDepTime}}$  and  $\Delta D_{\text{SchedArrTime}}$  are incrementally computed. These partial distance matrices are approximately of size  $2 \times (6 + 2) = 16$ , compared to the full matrices of size  $(6 + 2) \times (6 + 2) = 64$  for  $I \cup \Delta I$ . This approach ensures efficient and parallel computation of distances across all processors in  $\mathcal{P}$ .

### 4.3 Incremental Clusters and Conflicts

To incrementally generate clusters for each attribute  $A_j \in R_\Sigma$ , we use the incremental distance matrix  $\Delta D_j \in \Delta D$  obtained in Section 4.2 to call the algorithm  $\text{TAGC}(\Delta D_j, \Theta_j, d_{1,|I|+1}, d_{|I+\Delta I|,|I+\Delta I|})$  to generate incremental clusters  $\Delta C_j$  by progressively refining the cluster for each threshold in  $\Theta_j$ . Notably, to handle the incremental upper triangular matrix of  $\Delta D_j$ , we embed it within an overall matrix  $D_j \cup \Delta D_j$  of size  $|I + \Delta I| \times |I + \Delta I|$ , where  $\Delta D_j$  occupies the rows from 1 to  $|I + \Delta I|$  and columns from  $|I| + 1$  to  $|I + \Delta I|$ .

To detect incremental conflicts with the incremental clusters  $\Delta C$ , we extend the ASPT algorithm discussed in Section 3.3.2. By invoking  $\text{ASPT}(\varphi, \Delta C, i_{\min \max}, \mathcal{P})$ , the incremental sets  $\Delta R_{\leq}^p$ ,  $\Delta R_{>}^p$ , and  $\Delta R_r^p$  for each processor  $p \in \mathcal{P}$  are obtained from clusters  $\Delta C$  for all attributes in the given constraint  $\varphi$ ,

$$\begin{aligned} \Delta R_{\leq}^p = \{ & (A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in \Delta C_j[\theta], \\ & i \leq i_{\min \max}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_{\leq} \}, \end{aligned} \quad (19)$$

$$\begin{aligned} \Delta R_{>}^p = \{ & (A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in \Delta C_j[\theta], \\ & i \leq i_{\min \max}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_{>} \}, \end{aligned} \quad (20)$$

$$\begin{aligned} \Delta R_r^p = \{ & (A_j, \theta, \{(t_i, t_l) \mid (t_i, t_l) \in \Delta C_j[\theta], \\ & i \leq i_{\min \max}, (i - p) \bmod |\mathcal{P}| = 0\}) \mid (A_j, \theta) \in R_r \}. \end{aligned} \quad (21)$$

These sets store elements in the form  $(A_j, \theta, \Delta C_j^p[\theta])$ , where  $\Delta C_j^p[\theta]$  is the incremental tuple pair subset derived from the incremental cluster  $\Delta C_j[\theta]$  based on the attribute  $A_j$  and the threshold  $\theta$ . By calling  $\text{PFVP}(\Delta R_{\leq}^p, \Delta R_{>}^p, \Delta R_r^p)$  discussed in Section 3.4, the set of incremental violating tuple pairs  $\Delta C_\varphi^p$  for the constraint  $\varphi \in \Sigma$  can be identified.

**EXAMPLE 14.** Algorithm TAGC generates incremental clusters  $\Delta C_j$  using the incremental distance matrix  $\Delta D_j$  from algorithm IPDM. For example,  $\Delta D_{\text{Flight}}$  is divided across two processors,  $p_1$  and  $p_2$ , with a step size  $|\mathcal{P}| = 2$ . The processor  $p_1$  handles rows 1, 3, 5, and 7, while  $p_2$  handles rows 2, 4, and 6. Each processor generates clusters for its assigned rows, such as  $\Delta C_{\text{Flight}}^{p_1}[0] = \{(t_1, t_8), (t_3, t_7)\}$  and  $\Delta C_{\text{Flight}}^{p_2}[0] = \{(t_4, t_7)\}$ , forming the incremental clusters. In the algorithm ASPT,  $\Delta C$  is used to compute these sets  $\Delta R_{\leq}^p$ ,  $\Delta R_{>}^p$ , and  $\Delta R_r^p$  for each processor based on the constraint  $\varphi$ . For  $\varphi_1$ , these sets are divided among processors with the size  $|\mathcal{P}| = 2$ . For example,  $p_1$  handles  $\Delta R_{\leq}^{p_1}$ ,  $\Delta R_{>}^{p_1}$ , and  $\Delta R_r^{p_1}$ , while  $p_2$  handles  $\Delta R_{\leq}^{p_2}$ ,  $\Delta R_{>}^{p_2}$ , and  $\Delta R_r^{p_2}$ , each containing the corresponding tuple pair sets for its assigned rows.



For  $p_1$ ,  $\Delta R_{\leq}^{p_1}$  includes tuple pair sets for Flight and SchedDepTime with a threshold of 0, storing  $\{(\text{Flight}, 0, \{(t_1, t_8), (t_3, t_7)\}), (\text{SchedDepTime}, 0, \{(t_1, t_8), (t_3, t_7)\})\}$ . As  $R_{>}$  is empty in  $\varphi_1$ ,  $\Delta R_{>}^{p_1}$  is also empty. For the RHS attribute SchedArrTime in  $\varphi_1$ ,  $\Delta R_r^{p_1}$  stores  $\{(\text{SchedArrTime}, 0, \{(t_1, t_8)\})\}$ .

#### 4.4 Incrementally Detecting Errors

Similar to Section 3.5, we now present a complete algorithm IPED( $I, \Delta I, \Sigma, \mathcal{P}$ ) to incrementally identify the error set  $\Delta E$  for the instance  $I$  and the incremental instance  $\Delta I$  using processors  $\mathcal{P}$  in parallel. Firstly, the incremental distance matrices  $\Delta D$  for all attributes  $R_{\Sigma}$  in  $\Sigma$  are computed. For each attribute  $A_j \in R_{\Sigma}$ , based on the specific ranges ( $l = 1, s = |I| + 1, p = |I + \Delta I|, q = |I + \Delta I|$ ) described in Section 4.3, the algorithm TAGC( $\Delta D_j, \Theta_j, d_{ls}, d_{pq}$ ) is invoked to generate incremental clusters  $\Delta C_j$  using the incremental matrix  $\Delta D_j$  and the threshold set  $\Theta_j$ . For each constraint  $\varphi \in \Sigma$ , attributes are classified into  $R_{\leq}$ ,  $R_{>}$ , and  $R_r$  based on Formulas 6-8. With the reference row index  $i_{\text{minimax}}$ , the algorithm ASPT( $\varphi, \Delta C, i_{\text{minimax}}, \mathcal{P}$ ) is called to obtain  $\Delta R_{\leq}^p$ ,  $\Delta R_{>}^p$ , and  $\Delta R_r^p$  assigned to each processor  $p \in \mathcal{P}$  from clusters  $\Delta C$  for all attributes in the given constraint  $\varphi$ . Each processor invokes the algorithm PFVP( $\Delta R_{\leq}^p, \Delta R_{>}^p, \Delta R_r^p$ ) to compute the incremental violating tuple pair set  $\Delta C_{\varphi}^p$  for the constraint  $\varphi$ . Finally, each conflict cell is checked according to its probability  $P(t_i[A_i] \mid t_i[(A_j \cup X) \setminus A_i])$ , to identify the incremental errors. Since each processor handles its own subset of incremental conflict tuple pairs, the error identification step is also executed in parallel, ensuring the same level of efficiency as PED algorithm.

PROPOSITION 2. The complexity of the Algorithm IPED is  $O\left(\frac{|\Sigma| \cdot m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$ .

The proof of Proposition 2 can also be found in [1].

EXAMPLE 15. Algorithm IPDM generates the incremental distance matrices  $\Delta D$  for attributes Flight, SchedDepTime and SchedArrTime for  $\varphi_1$ , by constructing partial matrices for tuples  $(t_7, t_8)$  against  $(t_1, \dots, t_8)$ . Algorithm TAGC uses  $\Delta D_j$  to compute incremental clusters  $\Delta C_j$ . For example, for Flight, processors  $p_1$  and  $p_2$  generate  $\Delta C_{\text{Flight}}^{p_1}[0] = \{(t_1, t_8), (t_3, t_7)\}$  and  $\Delta C_{\text{Flight}}^{p_2}[0] = \{(t_4, t_7)\}$ . Algorithm ASPT creates  $\Delta R_{\leq}^p$ ,  $\Delta R_{>}^p$ , and  $\Delta R_r^p$ . For  $\varphi_1$  on  $p_1$ ,  $\Delta R_{\leq}^{p_1}$  contains attributes Flight and SchedDepTime with the same tuple pair set  $\{(t_1, t_8), (t_3, t_7)\}$ ,  $\Delta R_{>}^{p_1}$  is empty, and  $\Delta R_r^{p_1}$  contains SchedArrTime with tuple pair set  $\{(t_1, t_8)\}$ . Algorithm PFVP computes  $\Delta C_{\varphi}^p$ . For  $\varphi_1$  on  $p_1$ ,  $\Delta C_{\text{LHS}}^{p_1}$  is computed from  $\Delta R_{\leq}^{p_1}$ , and subtracted with  $\Delta R_r^{p_1}$  to obtain  $\Delta C_{\varphi_1}^{p_1} = \{(t_3, t_7)\}$ . Finally, algorithm IPED checks the conditional probabilities of cells in violating tuple pairs against their thresholds. For  $\varphi_1$ , since we have  $P(t_7[\text{SchedArrTime}] \mid t_7[\text{Flight}], t_7[\text{SchedDepTime}]) = 0.003 < \delta_{\text{SchedArrTime}} = 0.65$ , the cell  $t_7[\text{SchedArrTime}]$  is identified as an error. The final incremental error set is  $\Delta E = \{t_7[\text{SchedArrTime}]\}$ .

## 5 Experiment

The source code and data are available online [1].

### 5.1 Experimental Settings

Our algorithms are implemented in Python. Experiments are performed on a machine with Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz CPU and 512GB memory.

#### 5.1.1 Datasets. We use five datasets in experiments.

Flight provides details on the departure and arrival times of flights, derived from diverse data sources available on the internet. Owing to the inconsistency among different data sources, there are 18.29% erroneous values in 2,376 tuples over 6 attributes.

Table 2. Summary of constraint sets used in experiments, including composition ratio and construction details

| Constraint Set  | $ \Sigma $ | FD (% , count) | DD (% , count) | DC (% , count) | #Non-zero Thresholds | Generation Method | Sample Size |
|-----------------|------------|----------------|----------------|----------------|----------------------|-------------------|-------------|
| Flight-FD       | 6          | 100.0% (6)     | 0.0% (0)       | 0.0% (0)       | 0                    | [39]              | null        |
| Flight-DD       | 9          | 55.6% (5)      | 44.4% (4)      | 0.0% (0)       | 4                    | [31]              | all         |
| Flight-DD-all   | 163        | 8.6% (14)      | 91.4% (149)    | 0.0% (0)       | 149                  | [31]              | all         |
| Flight-DC       | 7          | 57.1% (4)      | 0.0% (0)       | 42.9% (3)      | 0                    | [48]              | all         |
| Hospital-FD     | 14         | 100.0% (14)    | 0.0% (0)       | 0.0% (0)       | 0                    | [39]              | null        |
| Hospital-DD     | 15         | 40.0% (6)      | 60.0% (9)      | 0.0% (0)       | 3                    | [31]              | all         |
| Hospital-DD-all | 28108      | 0.7% (199)     | 99.3% (27909)  | 0.0% (0)       | 27786                | [31]              | all         |
| Hospital-DC     | 14         | 85.7% (12)     | 0.0% (0)       | 14.3% (2)      | 0                    | [48]              | all         |
| Soccer-FD       | 5          | 100.0% (5)     | 0.0% (0)       | 0.0% (0)       | 0                    | [48]              | 10k         |
| Soccer-DD       | 5          | 60.0% (3)      | 40.0% (2)      | 0.0% (0)       | 2                    | [31]              | all         |
| Soccer-DD-all   | 22016      | 0.2% (45)      | 99.8% (21971)  | 0.0% (0)       | 21971                | [31]              | all         |
| Soccer-DC       | 8          | 25.0% (2)      | 0.0% (0)       | 75.0% (6)      | 0                    | [48]              | 10k         |
| MIMIC-FD        | 2          | 100.0% (2)     | 0.0% (0)       | 0.0% (0)       | 0                    | [48]              | 50k         |
| MIMIC-DD        | 4          | 50.0% (2)      | 50.0% (2)      | 0.0% (0)       | 2                    | [31]              | 50k         |
| MIMIC-DD-all    | 736        | 3.3% (24)      | 96.7% (712)    | 0.0% (0)       | 702                  | [31]              | 50k         |
| Plane-FD        | 4          | 100.0% (4)     | 0.0% (0)       | 0.0% (0)       | 0                    | [48]              | 50k         |
| Plane-DD        | 8          | 50.0% (4)      | 50.0% (4)      | 0.0% (0)       | 2                    | [31]              | 50k         |
| Plane-DD-all    | 106        | 5.7% (6)       | 94.3% (100)    | 0.0% (0)       | 100                  | [31]              | 50k         |

Table 3. Comparison of constraint generation time (s) between FastDD and IE

| Dataset        | $ \Sigma $ | FastDD [31] | IE [45] |
|----------------|------------|-------------|---------|
| Flight         | 163        | 4.271       | 8.945   |
| Hospital       | 28108      | 14.791      | n/a*    |
| Soccer-200k-5% | 22016      | 53235.156   | n/a*    |
| MIMIC          | 736        | n/a#        | n/a*    |
| Plane          | 106        | n/a#        | n/a*    |

\*: out of memory #: cannot terminate within one day

Hospital is a widely used dataset related to hospitals and healthcare service providers, comprising 1,000 tuples over 19 attributes, with 2.68% real-world errors.

Soccer [41] contains 200,000 tuples over 10 attributes, including the information about soccer players and their respective teams. We create different-sized variants by sampling subsets from the full Soccer dataset and name them accordingly. Since Soccer is originally clean, following existing studies [47], we inject 5% synthetic errors into each variant using BART [9]. Accordingly, we denote the full dataset with injected errors as Soccer-200k-5%, and its sampled variants as Soccer-10k-5%, Soccer-50k-5% and so on, where, for example, Soccer-10k-5% represents a 10,000-tuple subset of the full dataset with 5% errors.

MIMIC [2] is derived from the real-world *MIMIC-IV v3.1 ICU* dataset. We select the *outputevents* table with 9 attributes, which records charted outputs for ICU patients, and sample 800,000 tuples. The dataset contains 2.93% erroneous values.

Plane [3] consists of 776,067 tuples with 5 attributes, containing structured plane records collected from airline information systems. It includes 17.76% erroneous values.

**5.1.2 Competing Methods.** We compare our algorithms with eleven widely used error detection methods, including statistics-based Scare [49] and Boostclean [29], constraint-based Bigdancing [27], Holistic [18], HoloClean [43], Horizon [44], HEURISTIC and RELAXATION [47], OCQA [11–13], FastDD [31], and labeling-based Raha [35]. We can use the available code for Boostclean [4], HoloClean [5], Raha [6], HEURISTIC [7], and RELAXATION [7], as well as the reimplemented versions in [39] for Scare, Bigdancing, Holistic and Horizon. We additionally reimplement OCQA and adapt FastDD to support error detection under given DD constraints.

We evaluate two versions of our PED Algorithm. SPED executes all core procedures, namely distance matrix computation, clustering, conflict detection, and error determination, sequentially on a single processor ( $|\mathcal{P}| = 1$ ). In contrast, MPED distributes these tasks across 16 processors

Table 4. Error detection performance over Flight, Hospital, Soccer-200k-5%, MIMIC, and Plane datasets

| Dataset        | Metric    | SPED     | MPEP     | Scare  | Boostclean | Bigdancing | Holistic | Holoclean | Horizon  | Raha      | HEURISTIC RELAXATION | OCQA  | FastDD   |          |
|----------------|-----------|----------|----------|--------|------------|------------|----------|-----------|----------|-----------|----------------------|-------|----------|----------|
| Flight         | Precision | 0.783    | 0.783    | 0.813  | 0.000      | 0.455      | 0.461    | 0.230     | 0.177    | 0.746     | 0.297                | 0.295 | 0.220    | 0.346    |
|                | Recall    | 0.859    | 0.859    | 0.048  | 0.000      | 0.811      | 0.832    | 0.999     | 0.806    | 0.603     | 0.693                | 0.687 | 0.999    | 0.841    |
|                | F1        | 0.819    | 0.819    | 0.091  | 0.000      | 0.583      | 0.593    | 0.375     | 0.291    | 0.667     | 0.415                | 0.413 | 0.361    | 0.491    |
|                | Runtime   | 14.69    | 2.10     | 104.47 | 72.76      | 143.41     | 1741.99  | 22.79     | 2.75     | 10.03     | 0.60                 | 1.64  | 110.36   | 13.64    |
| Hospital       | Precision | 0.942    | 0.942    | 0.155  | 0.999      | 0.457      | 0.461    | 0.039     | 0.025    | 0.984     | 0.088                | 0.132 | 0.243    | 0.459    |
|                | Recall    | 0.802    | 0.802    | 0.104  | 0.181      | 0.843      | 0.794    | 0.855     | 0.640    | 0.611     | 0.747                | 0.764 | 0.726    | 0.794    |
|                | F1        | 0.866    | 0.866    | 0.124  | 0.306      | 0.593      | 0.583    | 0.075     | 0.048    | 0.754     | 0.157                | 0.225 | 0.364    | 0.582    |
|                | Runtime   | 9.94     | 2.13     | 471.29 | 19.50      | 156.06     | 953.64   | 55.72     | 1.25     | 22.32     | 0.49                 | 0.69  | 24.89    | 2.31     |
| Soccer-200k-5% | Precision | 0.981    | 0.981    | n/a#   | 0.999      | n/a#       | n/a#     | n/a*      | 0.007    | 0.560     | n/a*                 | n/a*  | 0.333    | 0.255    |
|                | Recall    | 0.949    | 0.949    | n/a#   | 0.090      | n/a#       | n/a#     | n/a*      | 0.919    | 0.300     | n/a*                 | n/a*  | 0.049    | 0.946    |
|                | F1        | 0.965    | 0.965    | n/a#   | 0.165      | n/a#       | n/a#     | n/a*      | 0.013    | 0.391     | n/a*                 | n/a*  | 0.086    | 0.402    |
|                | Runtime   | 65014.70 | 4825.37  | n/a#   | 1291.08    | n/a#       | n/a#     | n/a*      | 740.49   | 13830.837 | n/a*                 | n/a*  | 27452.15 | 34100.08 |
| MIMIC          | Precision | n/a#     | 0.503    | n/a*   | 0.970      | n/a#       | n/a*     | n/a*      | n/a#     | n/a*      | n/a*                 | n/a*  | 0.188    | n/a*     |
|                | Recall    | n/a#     | 0.622    | n/a*   | 0.002      | n/a#       | n/a*     | n/a*      | n/a#     | n/a*      | n/a*                 | n/a*  | 0.001    | n/a*     |
|                | F1        | n/a#     | 0.556    | n/a*   | 0.005      | n/a#       | n/a*     | n/a*      | n/a#     | n/a*      | n/a*                 | n/a*  | 0.002    | n/a*     |
|                | Runtime   | n/a#     | 54265.10 | n/a*   | 51337.27   | n/a#       | n/a*     | n/a*      | n/a#     | n/a*      | n/a*                 | n/a*  | 22979.08 | n/a*     |
| Plane          | Precision | n/a#     | 0.845    | n/a*   | n/a*       | n/a#       | n/a#     | 0.240     | 0.157    | n/a*      | n/a*                 | n/a*  | 0.000    | n/a*     |
|                | Recall    | n/a#     | 0.865    | n/a*   | n/a*       | n/a#       | n/a#     | 0.999     | 0.480    | n/a*      | n/a*                 | n/a*  | 0.000    | n/a*     |
|                | F1        | n/a#     | 0.855    | n/a*   | n/a*       | n/a#       | n/a#     | 0.387     | 0.237    | n/a*      | n/a*                 | n/a*  | 0.000    | n/a*     |
|                | Runtime   | n/a#     | 7388.21  | n/a*   | n/a*       | n/a#       | n/a#     | 32502.82  | 70444.12 | n/a*      | n/a*                 | n/a*  | 48026.54 | n/a*     |

\*: out of memory #: cannot terminate within one day

( $|\mathcal{P}| = 16$ ). This setup allows us to evaluate the effectiveness of our parallel design by comparing MPED with its sequential counterpart, SPED.

**5.1.3 Constraint Sets.** We summarize all constraint sets used in experiments in Table 2. For each constraint set, we report: the total size, FD/DD/DC composition, number of non-zero-threshold dependencies, generation method and sample size (where “null” denotes the external source, “all” denotes the full dataset, and a numeric value indicates the sample size). We adopt the naming convention Dataset-Type, such as Flight-DD, to refer to small subsets of constraints used in experiments, and Dataset-DD-all to denote the full sets mined by FastDD [31]. Variants like Flight-DD-1 through Flight-DD-163 refer to subsets of increasing sizes used for Figure 7. FDs for Flight and Hospital are taken from [39]. For other datasets lacking predefined FDs (e.g., Soccer, MIMIC, Plane), we extract FD subsets using FastADC [48] as special cases of DCs. All DCs are selected subsets of constraints mined by FastADC [48]. For efficiency, we use the sampled variant Soccer-10k-5% to mine FDs and DCs from the Soccer dataset. All DDs are mined using FastDD and we compare its performance against IE [45]. Table 3 summarizes their efficiency in mining full DD sets—Flight-DD-all, Hospital-DD-all, etc. While both methods produce the same number of DDs on datasets where they succeed, FastDD completes execution on Flight, Hospital, and Soccer, whereas IE runs out of memory on Hospital and Soccer and only completes on Flight. This makes FastDD a more practical and scalable choice for generating diverse DD constraints in our experiments. However, on extremely large-scale datasets such as MIMIC and Plane, neither FastDD nor IE can finish within acceptable time or resource limits. In such cases, we adopt 50k-sample subsets to enable tractable DD discovery; as shown in Table 2, both MIMIC-DD-all and Plane-DD-all are constructed from such subsets.

**5.1.4 Evaluation Metrics.** To evaluate the error detection accuracy, we adopt the widely used Precision, Recall and F1-score (F1). Referring to the ground truth of each dataset, we determine  $tp$  - the correctly identified errors,  $fp$  - clean values that are falsely determined as errors, and  $fn$  - erroneous cells that are not marked as such. The evaluation metrics are computed as

$$Recall = \frac{tp}{tp + fn}, \quad Precision = \frac{tp}{tp + fp}, \quad F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

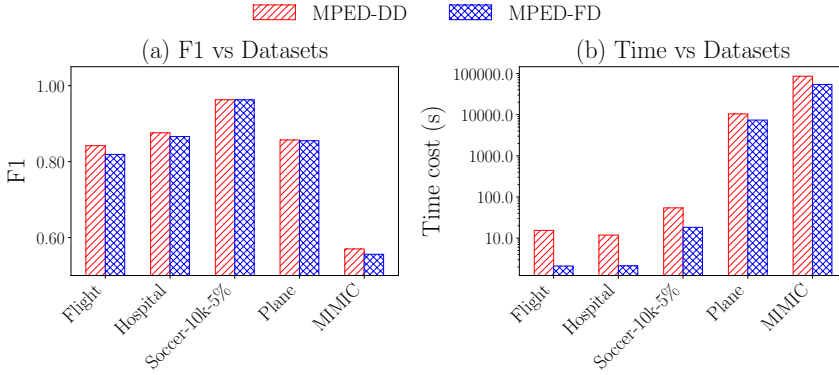


Fig. 5. DD vs. FD in error detection performance

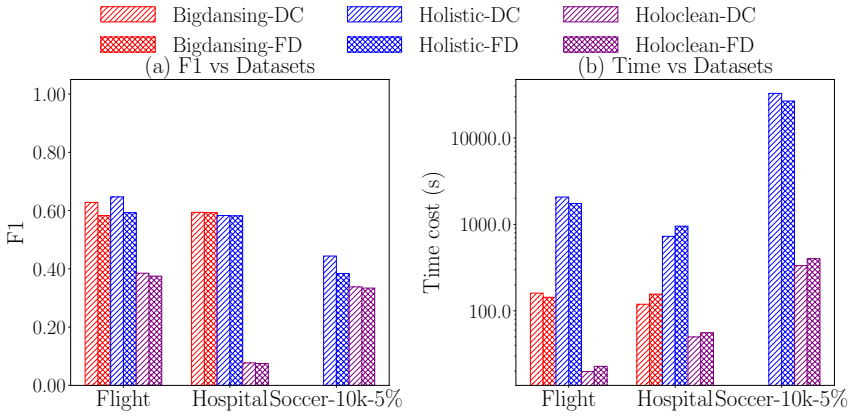


Fig. 6. DC vs. FD in error detection performance with DC-supporting methods

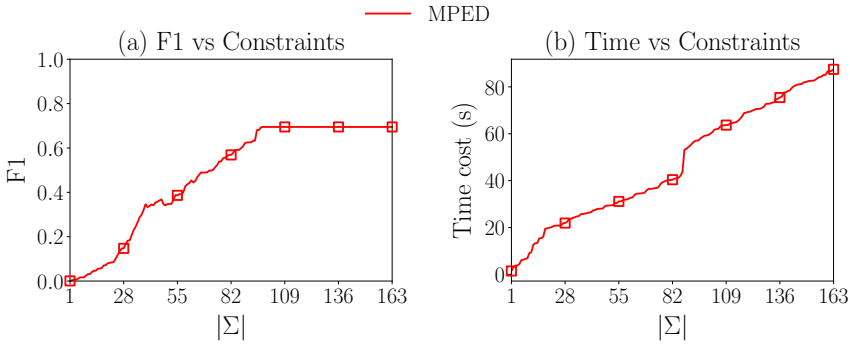


Fig. 7. Error detection performance with varying the number of Flight-DD-all

## 5.2 Comparative Evaluation with Baselines

We first compare error detection performance on the Flight, Hospital, Soccer-200k-5%, MIMIC, and Plane datasets in Table 4. Specifically, we use the following named constraint sets from Table 2: Flight-FD, Hospital-FD, Soccer-FD, MIMIC-FD, and Plane-FD. Scare and BoostClean are representative statistics-based error detection approaches. These methods rely on data distribution to detect outliers. While Scare and BoostClean typically achieve good precision by targeting outliers, their recall is low because relying solely on data distribution often misses errors, as discussed in the introduction. For example, BoostClean faces challenges on complex datasets like Flight and

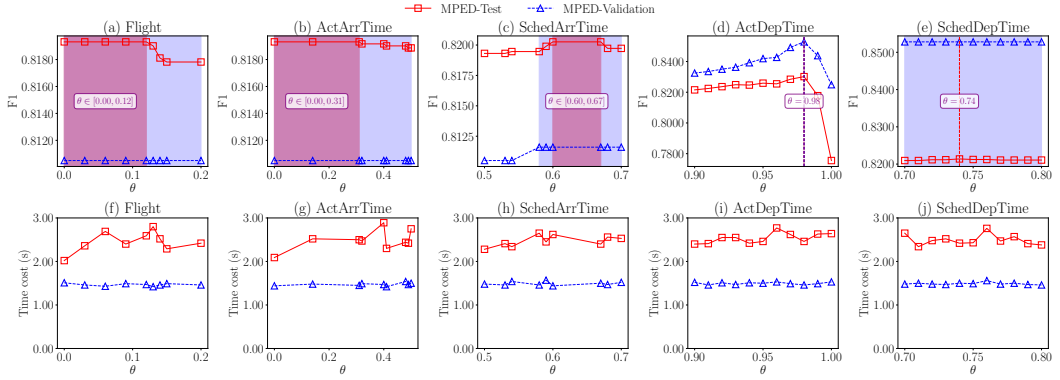


Fig. 8. Validation and sensitivity analysis of probability threshold selection across attributes on Flight

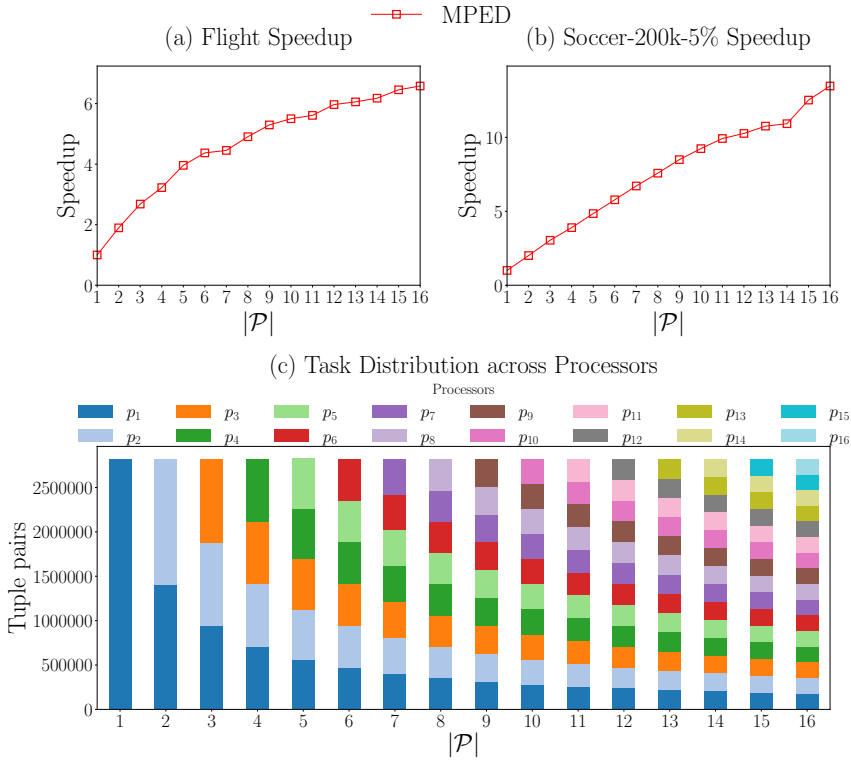


Fig. 9. Speedup and load balancing with increasing processors

runs out of memory on Plane due to costly feature extraction. Scare runs out of memory on MIMIC and Plane due to per-partition model training and large repair candidate space.

Constraint-based methods include BigDancing, Holistic, Holoclean, Horizon, HEURISTIC, RELAXATION, OCQA and FastDD, all of which leverage integrity constraints to detect errors. These methods benefit from integrity constraints, leading to higher accuracy on small datasets. However, they struggle on large-scale datasets due to scalability limitations. BigDancing and Holistic fail on Soccer-200k-5%, MIMIC, and Plane due to costly reasoning and repair. Holoclean runs out of memory on Soccer-200k-5% and MIMIC, and is slow on Plane. Horizon only supports simple FDs,

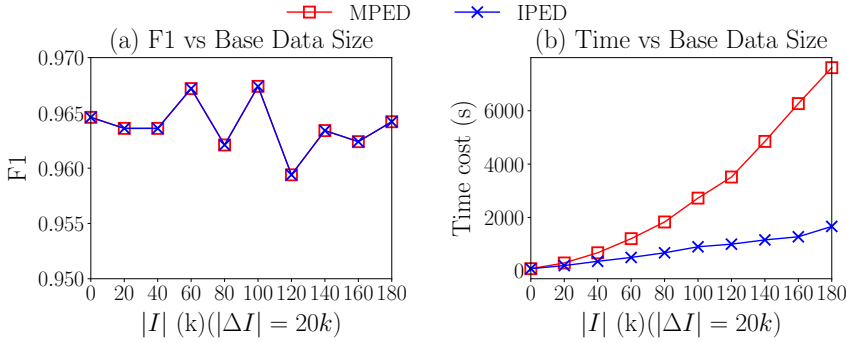


Fig. 10. Error detection performance with varying base data  $|I|$  and fixed 20k incremental batch  $|\Delta I|$  over Soccer

yielding low F1 and failing to finish within 24h on MIMIC. HEURISTIC and RELAXATION also run out of memory on large datasets due to the quadratic growth of conflict tuple pairs. OCQA is faster on MIMIC via sampling, but suffers from low recall. FastDD consumes excessive memory on MIMIC and Plane due to the explosive growth of diff-sets, and shows lower F1 performance due to its minimality-based strategy.

Raha combines multiple error detection techniques (e.g., dBoost [40] and KATARA [19]) with manual labeling to achieve strong performance. However, its reliance on just twenty labeled tuples limits its ability to detect diverse error patterns, especially on the Soccer-200k-5% dataset. As shown in Table 4, Raha runs out of memory on MIMIC and Plane, since it generates high-dimensional feature vectors for each cell based on numerous error detection strategies, leading to memory explosion. In contrast, our algorithm MPED successfully scales to both MIMIC and Plane. While its runtime on MIMIC is higher than BoostClean and OCQA, it achieves better memory efficiency and consistently higher F1 scores than baselines.

### 5.3 Influence of Constraint Sets

We conduct experiments on the diversity of DDs in Figure 5, by comparing the performance of our MPED algorithm using Flight-DD vs. Flight-FD, Hospital-DD vs. Hospital-FD and so on across all five datasets. Results show that although more expressive constraints (DDs) can enhance error detection quality and outperform baselines, they incur substantial runtime and scalability costs, highlighting the trade-off between effectiveness and efficiency.

Figure 6 reuses the same FD results for BigDancing, Holistic, and Holoclean (i.e., on Flight-FD, Hospital-FD, Soccer-FD), and additionally compares them with their performance under DC constraints (Flight-DC, Hospital-DC, Soccer-DC), only for these DC-compatible methods. We exclude MIMIC-DC and Plane-DC as these methods fail to produce results on the MIMIC and Plane datasets. As shown in Figure 6(a), applying DCs generally leads to higher F1 scores than FDs, indicating their superior expressiveness in capturing complex error patterns. However, Figure 6(b) shows that using more expressive DCs usually results in longer runtimes, demonstrating the trade-off between constraint expressiveness and computational efficiency.

Figure 7 illustrates the results of our parallel method as the number of constraints  $|\Sigma|$  varies. Specifically, we vary the size of Flight-DD-all from Flight-DD-1 to Flight-DD-163, using 1, 2, 3, ..., 163 DD constraints, all mined by FastDD. As shown in Figure 7(a), while some DDs degrade performance or have no effect, increasing their number generally leads to higher F1 scores. Meanwhile, Figure 7(b) shows a clear increase in runtime with more constraints. These results highlight the trade-off between detection quality and efficiency. In practical applications, existing methods [31] can be

utilized to reason about rules and eliminate redundant semantics, although this is beyond the scope of our current study.

In practice, we can use the validation set to select the probability thresholds by tuning them to maximize the error detection accuracy. To validate the effectiveness of this strategy, we conduct the experiment on the Flight dataset using Flight-FD in Figure 8. For each attribute, the red curve shows how the F1 score changes on the full dataset (used as test set) as the probability threshold  $\delta_j$  varies. The red shaded area marks the true optimal threshold range. Our strategy selects thresholds using a small validation set of 500 manually labeled tuples; the resulting F1 scores and the selected threshold ranges are represented by blue curves and blue shaded regions. Darker overlapping areas indicate agreement between the selected and the true optimal thresholds, and vertical dashed lines denote exact matches. Across all attributes, the selected thresholds fall within the true optimal ranges, demonstrating the effectiveness and robustness of our threshold selection strategy.

#### 5.4 Scalability with Parallelization and Incremental Processing

As reported in Table 4, our parallel version MPED ( $|\mathcal{P}| = 16$ ) achieves significant runtime improvements over the sequential version SPED ( $|\mathcal{P}| = 1$ ), with speedups of  $7\times$  on Flight,  $5\times$  on Hospital, and  $13\times$  on Soccer-200k-5%. In contrast, SPED fails to complete execution on MIMIC and Plane, while MPED successfully processes both datasets within reasonable time. Overall, MPED demonstrates strong scalability and consistently achieves the best performance in both efficiency and accuracy across all datasets, significantly outperforming all baselines. Additionally, we observe that varying the error rate has negligible impact on detection results, and thus omit further discussion.

To further evaluate the parallel scalability of our method, we present Figure 9, where subfigures (a) and (b) show the speedup achieved with 1–16 processors on Flight and Soccer-200k-5%, using Flight-FD and Soccer-FD, respectively. The results demonstrate a clear trend of increasing speedup with more processors, validating the effectiveness of our parallel design. Figure 9(c) visualizes the task distribution, confirming that workloads are evenly balanced across processors.

Figure 10 reports the accuracy and time costs of our parallel detection methods MPED and IPED with incremental computation strategies, on the Soccer dataset using Soccer-FD. As illustrated in this figure, the red line (MPED) indicates the runtime of rerunning the entire dataset after adding a fixed 20k incremental batch to varying base sizes. In contrast, the blue line (IPED) shows the runtime of processing only the fixed 20k incremental batch using our incremental algorithm. As the ratio between the base dataset and the increment increases, the runtime difference between IPED and MPED becomes more pronounced, demonstrating the substantial runtime savings of our incremental algorithm under large base-to-increment ratios. Incremental algorithms are typically designed to process small-scale updates over an existing large dataset, avoiding the overhead of full recomputation [23]. Therefore, we conduct the experiments in Figure 10 under realistic conditions where incremental changes are relatively small compared to the already processed data.

## 6 Related Work

Statistics-based methods detect errors by leveraging statistical properties. ERACER [37] uses relational dependency networks and belief propagation to infer missing values and detect errors based on cross-tuple correlations. Scare [49] employs a Bayesian model to detect and repair errors by modeling attribute co-occurrence and inferring the most likely clean values. dBoost [40] combines techniques like histograms and Gaussian models to identify outliers. Auto-Detect [25] leverages co-occurrence statistics mined from large table corpora to model value dependencies across attributes and identify unlikely value combinations as potential errors, while SrFn [46] detects misplaced values by swapping attributes.

Constraint-based methods focus on cells that violate integrity constraints, such as FD [36], CFD [22], DC [16], DD [45], SC [50], and forbidden itemsets [42]. Many methods adopt the minimum change principle [20] to minimize excessive error detection and preserve data consistency with as few modifications as possible. For example, BigDancing [27] allows users to define data quality rules to compute minimal error sets, and Holistic [17] iteratively identifies errors under denial constraints. HoloClean [43] combines constraints and statistics for error detection, adhering to the minimum change principle. Horizon improves efficiency with a linear-time algorithm based on functional dependencies, but its limited expressiveness reduces its practicability. Our approach differs fundamentally from FastDD [31] in problems, constraint usage, and methodologies. (1) As for problems, FastDD aims to discover all the minimal DDs, while we focus on detecting erroneous cells, without relying on the minimum change principle. (2) Although both approaches use DDs, their usage differs significantly. FastDD focuses on generating a complete set of minimal DDs, whereas our method assumes a set of predefined DDs and performs error detection accordingly. (3) FastDD constructs diff-sets and applies set-cover enumeration to discover all valid and minimal DDs, while our method performs parallel distance matrix construction, clustering, and conflict detection, followed by probabilistic error identification based on conditional likelihood. OCQA (Operational Consistent Query Answering) [11–13] is a framework that assigns probabilities to repair results by constructing a Markov chain. The correctness of each query answer is associated with a confidence score derived from the probability of the corresponding repair.

With the rise of AI, labeling-based methods [24, 30, 34, 38] are increasingly applied to error detection. ActiveClean [30] iteratively repairs records based on user labels to optimize data and models, prioritizing impactful records. Raha [34] extends this with a semi-supervised approach, reducing user involvement through automatic feature generation and clustering. HoloDetect [24] builds on Raha with data augmentation to address class imbalance but heavily relies on augmentation. ED2 [38] further improves HoloDetect by employing a two-stage active learning strategy to reduce labeling costs.

## 7 Conclusion

In this work, recognizing the limitations of existing methods, we design error detection strategies based on Bayesian statistics, guided by integrity constraints. To ensure the effectiveness and efficiency of our methods when dealing with large-scale data, we (1) formalize the error detection problem using Bayesian statistics under integrity constraints, allowing for probabilistic modeling of attribute dependencies and uncertainty; (2) devise parallel error detection algorithms with guaranteed parallel scalability; (3) design incremental error detection methods to further address new massive data that may arise at any time, thereby avoiding the need to rerun the entire algorithm on all data. Experiments on five datasets, namely Flight, Hospital, Soccer, MIMIC and Plane, demonstrate that our method consistently outperforms a strong baseline in both error detection accuracy and time efficiency across diverse data domains. We also conduct evaluation experiments on key parameters to assess their impact and guide future improvements. In summary, our method is able to detect errors more accurately while significantly reducing runtime, showing clear advantages in both effectiveness and scalability.

## Acknowledgements

We are deeply grateful for the valuable suggestions and guidance provided by the anonymous reviewers. This work is supported in part by the National Natural Science Foundation of China (62302241, 62372252, 72342017, 92267203, 62232005, 62021002), the National Key Research and Development Plan (2024YFB3311901).



## References

- [1] <https://github.com/twinklelittlestars/PED>
- [2] <https://mimic.mit.edu/>
- [3] <https://lunadong.com/fusiondatasets>
- [4] <https://github.com/sjyk/activedetect>
- [5] <https://github.com/HoloClean/holoclean>
- [6] <https://github.com/BigDaMa/raha>
- [7] <https://github.com/densitysrepair/densitysrepair>
- [8] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting Data Errors: Where are we and what needs to be done? *PVLDB* 9, 12 (2016), 993–1004. doi:10.14778/2994509.2994518
- [9] Patricia C. Arocena, Boris Glavic, Giansalvatore Mecca, Renée J. Miller, Paolo Papotti, and Donatello Santoro. 2015. Messing Up with BART: Error Generation for Evaluating Data-Cleaning Algorithms. *PVLDB* 9, 2 (2015), 36–47. doi:10.14778/2850578.2850579
- [10] George Beskales, Ihab F. Ilyas, and Lukasz Golab. 2010. Sampling the Repairs of Functional Dependency Violations under Hard Constraints. *PVLDB* 3, 1 (2010), 197–207. doi:10.14778/1920841.1920870
- [11] Marco Calautti, Leonid Libkin, and Andreas Pieris. 2018. An Operational Approach to Consistent Query Answering. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, Houston, TX, USA, June 10-15, 2018*, Jan Van den Bussche and Marcelo Arenas (Eds.). ACM, 239–251. doi:10.1145/3196959.3196966
- [12] Marco Calautti, Ester Livshits, Andreas Pieris, and Markus Schneider. 2022. Uniform Operational Consistent Query Answering. In *PODS '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Leonid Libkin and Pablo Barceló (Eds.). ACM, 393–402. doi:10.1145/3517804.3526230
- [13] Marco Calautti, Ester Livshits, Andreas Pieris, and Markus Schneider. 2024. Combined Approximations for Uniform Operational Consistent Query Answering. *Proc. ACM Manag. Data* 2, 2 (2024), 99. doi:10.1145/3651600
- [14] Loredana Caruccio, Vincenzo Deufemia, Felix Naumann, and Giuseppe Polese. 2020. Discovering Relaxed Functional Dependencies Based on Multi-Attribute Dominance. *IEEE Trans. Knowl. Data Eng.* 33, 9 (2020), 3212–3228. doi:10.1109/TKDE.2020.2974602
- [15] Jan Chomicki and Jerzy Marcinkowski. 2005. Minimal-change integrity maintenance using tuple deletions. *Inf. Comput.* 197, 1-2 (2005), 90–121. doi:10.1016/j.ic.2004.04.007
- [16] Xu Chu, Ihab F Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *Proceedings of the VLDB Endowment* 6, 13 (2013), 1498–1509.
- [17] Xu Chu, Ihab F Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 458–469.
- [18] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *ICDE*. 458–469. doi:10.1109/ICDE.2013.6544847
- [19] Xu Chu, John Morcos, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Nan Tang, and Yin Ye. 2015. KATARA: A Data Cleaning System Powered by Knowledge Bases and Crowdsourcing. In *SIGMOD, Melbourne, Victoria, Australia, May 31 - June 4, 2015*. 1247–1261. doi:10.1145/2723372.2749431
- [20] Wenfei Fan. 2009. Constraint-Driven Database Repair.
- [21] Wenfei Fan, Hong Gao, Xibei Jia, Jianzhong Li, and Shuai Ma. 2011. Dynamic constraints for record matching. *VLDB J.* 20, 4 (2011), 495–520. doi:10.1007/s00778-010-0206-6
- [22] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 1–48.
- [23] Wenfei Fan, Chao Tian, Yanghao Wang, and Qiang Yin. 2021. Parallel Discrepancy Detection and Incremental Detection. *Proc. VLDB Endow.* 14, 8 (2021), 1351–1364. doi:10.14778/3457390.3457400
- [24] Alireza Heidari, Joshua McGrath, Ihab F. Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: Few-Shot Learning for Error Detection. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 829–846. doi:10.1145/3299869.3319888
- [25] Zhipeng Huang and Yeye He. 2018. Auto-Detect: Data-Driven Error Detection in Tables. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 1377–1392. doi:10.1145/3183713.3196889
- [26] James Joyce. 2003. Bayes' theorem. (2003).
- [27] Zuhair Khayyat, Ihab F Ilyas, Alekh Jindal, Samuel Madden, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Si Yin. 2015. Bigdancing: A system for big data cleansing. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 1215–1230.
- [28] Nick Koudas, Avishek Saha, Divesh Srivastava, and Suresh Venkatasubramanian. 2009. Metric Functional Dependencies. In *ICDE*. 1275–1278. doi:10.1109/ICDE.2009.219

- [29] Sanjay Krishnan, Michael J. Franklin, Ken Goldberg, and Eugene Wu. 2017. BoostClean: Automated Error Detection and Repair for Machine Learning. *CoRR* abs/1711.01299 (2017). arXiv:1711.01299 <http://arxiv.org/abs/1711.01299>
- [30] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: interactive data cleaning for statistical modeling. *Proc. VLDB Endow.* 9, 12 (Aug. 2016), 948–959. doi:10.14778/2994509.2994514
- [31] Shulei Kuang, Honghui Yang, Zijing Tan, and Shuai Ma. 2024. Efficient Differential Dependency Discovery. *Proc. VLDB Endow.* 17, 7 (2024), 1552–1564. doi:10.14778/3654621.3654624
- [32] Dennis Victor Lindley. 1972. *Bayesian statistics: A review*. SIAM.
- [33] Daniel Lowd and Pedro Domingos. 2005. Naive Bayes models for probability estimation. In *Proceedings of the 22nd international conference on Machine learning*. 529–536.
- [34] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 865–882. doi:10.1145/3299869.3324956
- [35] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *SIGMOD*. 865–882. doi:10.1145/3299869.3324956
- [36] Panagiotis Mandros, Mario Boley, and Jilles Vreeken. 2017. Discovering reliable approximate functional dependencies. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 355–363.
- [37] Chris Mayfield, Jennifer Neville, and Sunil Prabhakar. 2010. ERACER: a database approach for statistical inference and data cleaning. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6–10, 2010*, Ahmed K. Elmagarmid and Divyakant Agrawal (Eds.). ACM, 75–86. doi:10.1145/1807167.1807178
- [38] Felix Neutatz, Mohammad Mahdavi, and Ziawasch Abedjan. 2019. ED2: A Case for Active Learning in Error Detection. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (Beijing, China) (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 2249–2252. doi:10.1145/3357384.3358129
- [39] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proc. VLDB Endow.* 17, 10 (2024), 2617–2630. doi:10.14778/3675034.3675051
- [40] Clement Pit-Claudel, Zelda Mariet, Rachael Harding, and Sam Madden. 2016. Outlier detection in heterogeneous datasets using automatic tuple expansion. (2016).
- [41] Joeri Rammelaere and Floris Geerts. 2018. Explaining Repaired Data with CFDs. *PVLDB* 11, 11 (2018), 1387–1399. doi:10.14778/3236187.3236193
- [42] Joeri Rammelaere and Floris Geerts. 2019. Cleaning data with forbidden itemsets. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1489–1501.
- [43] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *PVLDB* 10, 11 (2017), 1190–1201. <http://www.vldb.org/pvldb/vol10/p1190-rekatsinas.pdf>
- [44] El Kindi Rezig, Mourad Ouzzani, Walid G. Aref, Ahmed K. Elmagarmid, Ahmed R. Mahmood, and Michael Stonebraker. 2021. Horizon: Scalable Dependency-driven Data Cleaning. *Proc. VLDB Endow.* 14, 11 (2021), 2546–2554. doi:10.14778/3476249.3476301
- [45] Shaoxu Song and Lei Chen. 2011. Differential Dependencies: Reasoning and Discovery. *ACM Trans. Database Syst.* 36, 3 (2011), 16:1–16:41. doi:10.1145/2000824.2000826
- [46] Yu Sun, Shaoxu Song, Chen Wang, and Jianmin Wang. 2020. Swapping Repair for Misplaced Attribute Values. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20–24, 2020*. IEEE, 721–732. doi:10.1109/ICDE48307.2020.00068
- [47] Yu Sun, Shaoxu Song, and Xiaojie Yuan. 2024. From Minimum Change to Maximum Density: On Determining Near-Optimal S-Repair. *IEEE Trans. Knowl. Data Eng.* 36, 2 (2024), 627–639. doi:10.1109/TKDE.2023.3294401
- [48] Renjie Xiao, Zijing Tan, Haojin Wang, and Shuai Ma. 2022. Fast Approximate Denial Constraint Discovery. *Proc. VLDB Endow.* 16, 2 (2022), 269–281. doi:10.14778/3565816.3565828
- [49] Mohamed Yakout, Laure Berti-Équille, and Ahmed K. Elmagarmid. 2013. Don't be SCARED: use Scalable Automatic REpairing with maximal likelihood and bounded changes. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22–27, 2013*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). ACM, 553–564. doi:10.1145/2463676.2463706
- [50] Jing Nathan Yan, Oliver Schulte, MoHan Zhang, Jiannan Wang, and Reynold Cheng. 2020. Scoded: Statistical constraint oriented data error detection. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 845–860.

Received January 2025; revised April 2025; accepted May 2025